AOSP 进阶丛书 · 第七册 · ANDROID 13 (API 33)

系统工程师面试题集

82 道系统级面试题：Binder、init/Zygote 启动、Input 事件分发、AMS/WMS、ROM · BSP · 刷机与综合场景——每题三件套：问题、带源码锚点的深度解答、面试官追问链。

源码基线：AOSP Android 13 (TP1A · SDK 33) · 行号全部实测核验

目录

前言：把六册读成你的话术 2

第一章 Binder（15 题） 2

第二章 启动主线：init / Zygote（15 题） 5

第三章 Input 事件分发（10 题） 9

第四章 AMS 与进程管理（12 题） 11

第五章 WMS 与渲染（10 题） 14

第六章 ROM / BSP / 刷机（12 题） 16

第七章 综合场景题（8 题） 18

第八章 深水区题（20 题） 20

第九章 补全题（10 题） 25

第十章 渲染深潜题（10 题） 27

第十一章 后台与电源题（10 题） 29

第十二章 网络栈题（10 题） 32

第十三章 存储与加密题（10 题） 34

第十四章 媒体管线题（10 题） 37

第十五章 SystemUI 与通知题（10 题） 39

第十六章 HAL 实战题（10 题） 42

第十七章 构建系统题（10 题） 44

第十八章 车机 AAOS 题（10 题） 47

第十九章 无障碍体系题（10 题） 49

第二十章 CTS 认证题（10 题） 51

第二十一章 Telephony 题（10 题） 53

附录 面试策略五条 55

本册速览

232 题十六族：覆盖十九册全部内容的面试题集
每题三件套：问题 + 带源码锚点的深度解答 + 追问链
每题标注〔册・章〕可回查十八册正文
含多个终极串联题（横跨多册的综合叙事）
用法：先自答再看解答；追问链务必脑内走一遍

前言：把六册读成你的话术

本册是前六册的"输出化训练"：**232 道系统级面试题，每题三件套——问题、深度解答（带源码锚点）、追问链（面试官的下一刀）**。用法建议：先自答再看解答；追问链务必在脑内走一遍——面试的差距不在第一问，在第三问。

解答里的行号与前六册一致（本树 Android 13 实测）。标注〔册·章〕表示出处，可回查。

第一章 Binder（15 题）

Q1.1 Binder 为什么只拷贝一次数据?

【深度解答】接收进程对 /dev/binder mmap 一块 1MB-2 页只读区（ProcessState.cpp:47/503），内核把该区同时映射进内核地址空间。发送时驱动的 binder_transaction 里一次 copy_from_user 直写这块双重映射区，接收方读自己的映射即得数据，省掉 copy_to_user。配套设计：PROT_READ 最小权限、MAP_NORESERVE 省内存、MAP_PRIVATE 隔离回写。对照：管道两次拷贝、共享内存零次但无安全/同步语义——Binder 是"每微秒买最多语义"。〔册3·一〕

【追问链】→ 为什么不全部用共享内存？→ Binder 的能力传递（flat_binder_object 翻译句柄）怎么做？→ AIDL 传大位图走什么路径（fd+共享内存）？

Q1.2 Binder 的安全模型为什么可信?

【深度解答】身份在内核采集：binder_transaction 把 current->cred 的 euid/pid 写进事务（sender_euid），服务端 executeCommand 装载为 mCallingUid/mCallingPid（IPCThreadState.cpp:1269-1273），任何用户态伪造无效。第二层是 SELinux 上下文随事务携带（BR_TRANSACTION_SEC_CTX，:1227）。第三层是 servicemanager 名录准入（canAdd/canFind）与 VINTF 声明一致性。三层职责：名录管"找得到"，驱动管"你是谁"，服务管"你能干什么"。〔册3·五/七〕

【追问链】→ clearCallingIdentity 为什么存在、忘恢复会怎样？→ 普通 App 能伪造 getCallingUid 吗？→ SELinux 拦截发生在哪一层？

Q1.3 一次同步 Binder 调用的完整十跳?

【深度解答】①getService（ActivityManager.java:4675，Singleton 缓存）②Proxy.transact→BinderProxy.transactNative(:614) ③JNI android_os_BinderProxy_transact(:1387) ④IPCThreadState.transact(:706)→writeTransactionData(BC_TRANSACTION :725) ⑤talkWithDriver ioctl(:997) ⑥驱动 binder_transaction：验句柄/采身份/一次拷贝/入队唤醒 ⑦服务端 executeCommand(:1147) 收 BR_TRANSACTION_SEC_CTX ⑧JavaBBinder→Binder.execTransact→Stub.onTransact ⑨业务执行 BC_REPLY ⑩客户端 BR_REPLY(:939) 零拷贝装 reply 返回。〔册3·九〕

【追问链】→ oneway 在哪一跳分岔（flags 一个比特）？→ 嵌套调用（服务端再调别人）如何不错乱（transaction_stack）？→ 十跳哪一跳最容易成为性能瓶颈（⑦调度唤醒）？

Q1.4 oneway 的真实语义三个误区?

【深度解答】误区一"立即执行"：oneway 到 BR_TRANSACTION_COMPLETE（:921）即返回，只保证"内核已收件"；误区二"天然并发"：同一目标的 oneway 在驱动内单队列保序（async_todo），跨进程才可能乱；误区三"随便发"：async 空间约为映射区一半，塞满即 BR_FAILED_REPLY——大包 oneway 翻车的根源。Android 13 另有泛洪检测 BR_ONEWAY_SPAM_SUSPECT(:915)。〔册3・三/十〕

【追问链】→ 心跳类 oneway 该怎么设计（分包/降频）？→ oneway 大数据的替代方案？→ AIDL 里哪些方法天然该 oneway（回调/通知类）？

Q1.5 Binder 线程池 15+1 的供给制怎么运转?

【深度解答】主线程由 startThreadPool 进入（App 在 zygoteInit→onZygoteInit:92-96），BC_ENTER_LOOPER；驱动按并发压力发 BR_SPAWN_LOOPER(:1373)→spawnPooledThread→BC_REGISTER_LOOPER，上限 BINDER_SET_MAX_THREADS（普通 15，system_server 31，SystemServer.java:435/854）。线程退出 BC_EXIT_LOOPER。排障铁律：线程打满+全忙=新同步调用排队（表现为对端"卡"而非自身卡）。〔册3・十〕

【追问链】→ 为什么 system_server 翻倍？→ binder 线程和 Handler 线程的关系（事务到 ApplicationThread 再转主线程 Looper）？→ ANR 时 binder 栈长什么样（debuggerd 三类栈型）？

Q1.6 servicemanager 为什么零线程池、仅 oneway?

【深度解答】名录是全机寻址根，必须极稳极简：setThreadPoolMaxThreadCount(0)（main.cpp:125）杜绝任何客户端把名录挂住；FATAL_IF_NOT_ONEWAY(:126) 从机制上禁同步调用；Looper 单线程 pollAll(-1) 天然串行化注册与查询。自举：addService("manager", this)(:129)+becomeContextManager(:134) 拿下句柄 0。〔册3・七〕

【追问链】→ 句柄 0 为什么不能给别的服务？→ servicemanager 死了会怎样（critical 连带重启整类服务）？→ 三域（binder/hwbinder/vndbinder）为什么隔离？

Q1.7 注册服务时的 linkToDeath 有什么用?

【深度解答】addService 的五道关卡之一（ServiceManager.cpp:325）：注册即挂死亡通知，服务进程死→驱动向 ref 广播 BR_DEAD_BINDER→servicemanager 自动除名，名录永无"僵尸服务"。同一机制的著名客户：AMS 的 AppDeathRecipient（ProcessList.java:5056）监 App 进程死、清理组件与窗口。〔册3・七/十〕

【追问链】→ linkToDeath 在客户端怎么用（DeathRecipient+unlink 必须成对）？→ BinderProxy 泄漏与死亡通知的关系（gNumProxies 告警）？→ system_server 重启后 App 侧如何自愈（缓存失效重取）？

Q1.8 Parcel 为什么是"事务即域"?

【深度解答】服务端事务里的 data/reply 由 binder 映射区背书，事务返回缓冲即被回收（freeBuffer）。跨事务保存 Parcel=持有将被复用的内核内存，内容随时被下一事务覆盖。三类易错：异步线程延迟读、缓存 Parcel、子线程复用——症状都是"数据偶尔变花"。〔册3 · 六〕

【追问链】→ 要跨事务传大数据怎么办（fd/共享内存/分片）？→ writeStrongBinder 传的到底是什么（flat_binder_object 二态）？→ aidl 的 in/out/inout 与 Parcel 生命周期关系?

Q1.9 BinderProxy 与 BpBinder 为什么要两层?

【深度解答】native BpBinder 持 32 位句柄、走 mIn/mOut 命令流；Java BinderProxy 只是它的马甲（javaObjectForIBinder:749 建 BinderProxyNativeData 持 BpBinder，每 IBinder 每进程一个代理，gNumProxies 计数告警防泄漏）。好处：Java GC 世界与 native 生命周期解耦（DeathRecipientList 缝合）、JNI 边界窄化为指针存取。〔册3 · 八〕

【追问链】→ 同一服务取两次 getService 返回同一对象吗（缓存+getInstance 复用）？→ Binder 本地对象跨进程传回自己进程会怎样（javaObjectForIBinder 直返原对象）？→ 为什么 Java 侧不直接操作 ioctl?

Q1.10 AIDL 生成的 Stub/Proxy 结构与协议保护?

【深度解答】Stub.onTransact 按 code 分发（TRANSACTION_x=FIRST_CALL_TRANSACTION +n），入口 enforceInterface 校验 DESCRIPTOR；Proxy 持 mRemote（BinderProxy），写 token→参数→transact→读 reply。手写骨架三十行（册3 §8.3）覆盖 90% 场景；稳定 AIDL 由版本目录（aidl_api/1,2…）冻结，跨分区接口按 Stability/VINTF 等级约束演化。〔册3 · 八〕

【追问链】→ in/out 参数在生成代码里的差异？→ 两个进程 DEscriptor 不一致会怎样（SecurityException/坏包）？→ rust/ndk backend 与 java backend 互通吗（同一接口多后端）？

Q1.11 冻结进程的 Binder 行为（12+）?

【深度解答】缓存 App 被 cgroup freezer 冻结：CPU 全停、发往它的同步事务立即 BR_FROZEN_REPLY（waitForResponse:931）快速失败而非挂起超时；oneway 与广播默认 deferred（解冻补投）。设计动机：把"冻结"从模糊语义变成协议级确定行为，后台冻结才敢默认开。〔册3 · 十〕

【追问链】→ 冻结时它发出的调用呢？→ 谁冻结谁解冻（OomAdjuster→cgroup；AMS 解冻窗口期）？→ FROZEN_REPLY 业务侧如何兜底（重试/退避）？

Q1.12 嵌套 Binder 调用如何支持"递归"?

【深度解答】每个 binder 线程有 transaction_stack：A 调 B，A 线程压栈等待；B 处理中调 C，B 的服务线程以调用方身份压栈；C 回 B、B 回 A，逐层弹栈——调用链跨三个进程而不乱，靠的是"栈随线程走"。这也是 binder 线程"双面性"（能服务也能外呼）的基础。〔册3 · 三〕

【追问链】→ A 等 B、B 反过来等 A 会怎样（死锁，system_server 为何禁同步外呼）？→ 服务端如何知道自己被嵌套调用（getCallingUid 即上跳身份）？→ oneway 有栈吗（无，async 队列）？

Q1.13 一次 Binder 调用大约多贵?

【深度解答】十跳耗时账本：组包微秒级、ioctl 1-2μs、驱动 5-20μs（含一次 copy）、目标唤醒受调度影响（数 μs~ms）、业务自定、回程对称。同机往返常识值 0.1-0.5ms。性能问题九成在唤醒调度与业务，协议本身极少瓶颈——perfetto binder 轨道的正确读法。〔册3・九〕

【追问链】→ 如何测量一次调用的真实开销（systrace 埋点/binder_calls_stats）？→ 批量化接口为什么是常见优化（一批一往返）？→ 缓存 BinderProxy 的收益边界?

Q1.14 Binder 驱动的核心数据结构关系?

【深度解答】五件套：binder_proc（进程，threads/nodes/refs 树）→ binder_thread（线程，transaction_stack）→ binder_node（服务端点，ptr/cookie 指宿主 BBinder）→ binder_ref（每进程侧句柄 desc）→ binder_buffer（接收区账本，async 半区独立）。用户态对照：node↔BBinder（JavaBBinder）、ref.desc↔BpBinder.mHandle、Parcel↔buffer 一段+offsets。〔册3・三〕

【追问链】→ App 拿到的"activity"到底是什么（system_server 里 AMS node 的 ref）？→ AMS 死了句柄怎么办（BR_DEAD_BINDER 广播）？→ 为什么 offsets 表与数据平面分离（对象槽扫描）？

Q1.15 hwbinder/vndbinder 与 binder 为什么三域?

【深度解答】Treble 把框架/硬件/厂商通信隔离成三个 binder 设备与三套 servicemanager：框架演进不污染 vendor 进程、vendor 崩溃不波及框架、SELinux 分别治理。AIDL HAL（Android 10+）已把 HAL 拉回主域复用工具链，但三域仍是现状（书3 §7 图）。〔册3・七〕

【追问链】→ 为什么不干脆合并成一张名录（版本/信任边界不同）？→ AIDL HAL 走 binder 还是 hwbinder（主域）？→ 判断一个服务在哪域的最快方法（service list 分段/dumpsys -l）？

第二章 启动主线：init / Zygote（15 题）

Q2.1 init 为什么分两阶段、两次 re-exec?

【深度解答】鸡生蛋问题：SELinux 策略与 rc 剧本都在 /system 分区，而第一阶段只拥有 ramdisk——先 FirstStageMain 挂分区（first_stage_mount.cpp:87）+switch_root，再 SetupSelinux（selinux.cpp:938）装载策略，最后 SecondStageMain（init.cpp:824）干活。re-exec 的意义：每个阶段只信任"自己亲手布置好的世界"，安全状态切换天然彻底。〔册2・二〕

【追问链】→ ramdisk 里的 init 和 system 的 init 是同一个二进制吗（是，main.cpp 四身份）？→ 不分两阶段会怎样（无策略可加载/服务无标签）？→ GKI 后 ramdisk 为何拆成 init_boot 分区？

Q2.2 init 为什么坚持单线程?

【深度解答】三个词：无锁（全局零 mutex，时序可推理——:829-832 注释明言 signalfd 之前禁起线程）、公平（ExecuteOneCommand 每轮一条命令，:94）、确定（同 rc 同顺序，bootchart 可度量）。代价：单条命令阻塞卡住 init（exec 命令因此走子进程+管道同步补偿）。对照 system_server 的并发世界——init 只服务"顺序的世界"。〔册2 · 五〕

【追问链】→ SIGCHLD 为什么用 signalfd 而非信号处理器？→ 主循环为什么先 reap 再处理其他 fd（防 ctl.start 竞态 :1025-1028）？→ 属性等待（wait_for_prop）为什么暂停推进命令？

Q2.3 late-init 触发瀑布的顺序为什么不可调换？

【深度解答】init.rc:516-547 的子触发器顺序即依赖序：early-fs（vold 起步）→fs/post-fs（挂载）→post-fs-data（/data 解密、persist 属性、apexd）→zygote-start（加密状态三选一触发）→boot（class_start）。把 zygote-start 提前：zygote 预加载要读 /data 的 dex/资源，且 ro.crypto.state 未定无法选触发分支——启动直接 fatal 或反复重启。〔册2 · 四〕

【追问链】→ 充电模式为什么跳过这条瀑布（charger 触发器 :979）？→ 属性触发器与事件触发器怎么配合（zygote-start 三条件）？→ 新增守护进程的 rc 该放哪个阶段？

Q2.4 zygote 为什么禁止起线程？

【深度解答】fork 多线程进程只保证调用方线程存活，其余线程连同持有的锁状态一起消失——子进程里这些锁永远不可能释放（POSIX 明确）。startZygoteNoThreadCreation（ZygoteInit.java:826）从源头杜绝；USAP 补水等原生操作也在受控点进行。衍生命题：App 的多线程是从 fork 之后才开始的。〔册2 · 六〕

【追问链】→ zygote 里的 binder 线程池什么时候起（fork 后子进程 onZygoteInit）？→ 为什么 preload 后要 gcAndFinalize（减小 COW 复制基数）？→ WebView 预加载为什么在 zygote（进程间共享）？

Q2.5 fork 而不是 exec 的原因？

【深度解答】exec 会重建地址空间，预加载的类/资源/ART 状态全部作废；fork+写时复制让 200MB 级框架世界"加载一次、千次复用"——App 秒起与全机内存水位都靠它。forkSystemServer（:683-792）硬编码出生证明：uid/gid 1000、13 项能力位图、nice-name system_server。〔册2 · 六〕

【追问链】→ fork 后子进程怎么降权到目标 uid（forkAndSpecialize 特化）？→ system_server 死了为什么要连累 zygote（critical 组全有或全无）？→ USAP 与普通 fork 的取舍？

Q2.6 runSelectLoop 的孵化流程？

【深度解答】ZygoteServer.java:425：poll（服务 socket+会话+USAP 管道）→processCommand 解析参数（uid/gid/ABI/caps/rlimits/seInfo）→Zygote.forkAndSpecialize(:347)→native 侧降权/关 fd/贴标签→子进程 return command(:562) 退出循环→zygoteInit(:980) 三步（commonInit/nativeZygoteInit 起 binder 池/applicationInit）→ActivityThread.main。父进程继续 poll，永续孵化。〔册2 · 六〕

【追问链】→ 谁向 zygote 发请求（AMS 的 ZygoteProcess 写 socket）？ → USAP 池空了会怎样（退化为普通 fork）？ → 32 位 App 由谁孵（辅 zygote，ABI 路由）？

Q2.7 preloaded-classes 清单是怎么来的、能改吗?

【深度解答】AOSP 用 tools/preload 从真实开机 trace 挖掘（类加载计数超阈值入选），是"框架最内核工作集"的活文档；Go 版走 --enable-lazy-preload（:855）延迟加载。厂商可裁剪——每删一行省一点内存、慢一点首帧，是内存-流畅权衡的最前线。〔册2 · 六/九〕

【追问链】→ 清单里挑一个类说明为什么全局需要它？ → lazy 模式下第一次 fork 前补加载的代价？ → 怎么验证裁剪效果（bootchart+内存基线对比）？

Q2.8 SystemServer 为什么分 Bootstrap/Core/Other 三批?

【深度解答】依赖拓扑的显式化：Bootstrap 建地基（Installer→AMS 占位→PMS 扫包→…），Core 放轻统计，Other 大合唱（WMS/IMS…按 boot phase 交错）。批内顺序同样是依赖序（书2 service-deps 图）；phase（100→1000）是全局进度条，服务在 onBootPhase 挂钩就位，避免"查无此人"。〔册2 · 七〕

【追问链】→ PMS 为什么必须 Bootstrap（AMS/Home 都等它）？ → AMS 先生 PMS 后熟是什么意思（占位 vs systemReady :2712/:2801）？ → 加新服务该进哪一批发在哪个 phase 挂钩?

Q2.9 boot phase 的 550 和 600 分别放行什么?

【深度解答】550（PHASE_ACTIVITY_MANAGER_READY）：AMS 就绪，系统内部可开始跨服务联动；600（PHASE_THIRD_PARTY_APPS_CAN_START）：三方 App 允许启动——Home 点火在 600 前后（startHomeActivity 链，ATMS:5874）。1000 是 BOOT_COMPLETED 广播之后。分层的意义：把"系统可用"拆成可观测的阶梯。〔册2 · 七/八〕

【追问链】→ sys.boot_completed 与 phase 1000 谁先（属性先于广播尾巴）？ → 开机动画退出的条件链（finishBooting 门闩 :5075）？ → 怎么观测当前 phase（dumpsys 各服务 onBootPhase 日志）？

Q2.10 开机完成的"权威标志"是什么?

【深度解答】sys.boot_completed=1（AMS.java:5149，finishBooting 内）——只有 AMS 有"框架就绪（Home 可启动、广播可送达）"的完整视角。init.svc.* 只证明服务在跑，dev.bootcomplete 是内核视角补充。三方 App 判定开机完成监听 ACTION_BOOT_COMPLETED 广播（有序，末位）。〔册2 · 八〕

【追问链】→ 为什么不 init 来设这个属性？ → BOOT_COMPLETED 广播被 ANR 拖住会怎样（有序链尾部等待）？ → 开机耗时怎么分段测（am 开机后 boot_time_event / perfetto）？

Q2.11 属性系统为什么读快写慢、且能这么早可用?

【深度解答】读：全进程 mmap 只读共享区，零系统调用（纳秒级）；写：socket 到 init 属性服务（property_service.cpp:527 handle_property_set_fd），SO_PEERCRED 凭证+SELinux 上下文仲裁（HandlePropertySet:481→PropertySet:176：合法性/ctl.* 分流/persist 持久化）。早可用

的原因：不依赖 ART/binder——init 第二阶段即建（PropertyInit:887）。〔册1・六/十〕

【追问链】→ ctl.start 与 setprop 的关系（前缀劫持为控制命令）？→ persist.* 何时落盘（/data/property，post-fs-data 后可写）？→ ro.* 为什么只读（设后拒绝 PREADONLY）？

Q2.12 ueventd 和 init 的关系？

【深度解答】同一二进制的另一身份（main.cpp:61 argv[0] 切换）：监听内核 uevent、按 ueventd.rc 建 /dev 节点与权限、固件加载（firmware_handler）。主流程 wait_for_coldboot_done（init.cpp:960）等它冷插完成才继续——所以"ueventd 先跑、init 等、再触发"。〔册2・二〕

【追问链】→ /dev 里没有 udev 为什么也能工作？→ 设备节点权限错了找哪个 rc？→ fastboot/recovery 下 ueventd 也跑吗（recovery 有独立 rc）？

Q2.13 A/B 分区与 AVB 如何配合保证升级安全？

【深度解答】A/B：双槽位（slot A/B）无缝升级，boot_control HAL 切槽、update_engine 写后台槽；AVB（external/avb）：vbmeta 哈希链校验 boot/system/vendor…，回滚索引防降级，bootloader 按锁状态与校验结果亮橙/黄/红/绿屏。失败自动回退旧槽——"升级永不 brick"的两层保险。〔册1・七/notes・07〕

【追问链】→ 动态分区（super）解决什么（分区弹性）？→ 虚拟 A/B 的 COW 快照怎么做到不占双倍空间？→ 解锁后哪些校验仍在（仅 dm-verity 提示减弱）？

Q2.14 boot.img 里有什么、GKI 改了什么？

【深度解答】经典 boot.img=kernel+ramdisk（mkbootimg 头：页大小/基地址/标签）；Android 13 起 init_boot 分区装通用 ramdisk，boot 只剩 GKI 内核——通用内核镜像+设备 ramdisk 分离，内核碎片化治理的关键一步。dtbo 单独分区放设备树覆盖。〔册1・七/notes・07〕

【追问链】→ boot header v4 变了什么？→ vendor_boot 与 init_boot 的分工？→ fastboot flash boot 与 boot_a/boot_b 的关系（当前槽）？

Q2.15 从按下电源键到 Launcher，你能白板画出几层？

【深度解答】验收级答案分九段：BootROM→bootloader（AVB 校验）→kernel→init 一阶段（挂分区）→selinux_setup→init 二阶段（属性/epoll/解析 rc→触发瀑布）→zygote（preload→forkSystemServer→runSelectLoop）→SystemServer（三批服务+phase 推进+systemReady）→Home/BOOT_COMPLETED。每段至少一个锚点（dmesg"init second stage started!"、boot_progress_* 事件、sys.boot_completed）。〔册2・一〕

【追问链】→ 每一段的"完成信号"各是什么？→ 砍掉 zygote 预加载开机快多少、代价是什么？→ 面试官最爱追问的三个数字（线程 15+1/31、映射 1MB-2页、超时 5s/10s/20s）记牢了吗？

第三章 Input 事件分发（10 题）

Q3.1 一次触摸从指尖到 onTouchEvent 的完整链路?

【深度解答】七段：①内核 evdev（/dev/input/eventX）②EventHub.getEvents（EventHub.cpp:1533，epoll+inotify）③InputReader.loopOnce（:68→:103→:110）翻译成 NotifyMotionArgs ④InputDispatcher.notifyMotion(:3975) 入队→dispatchOnce(:602)→dispatchOnceInnerLocked(:718) 选目标 ⑤InputChannel socketpair（InputChannel.java:109 openInputChannelPair，addWindow 时配对注册）投给焦点窗口 ⑥App 侧 WindowInputEventReceiver→enqueueInputEvent→InputStage 责任链（Native→PreIme→Ime→View→ViewRootImpl）⑦View.dispatchTouchEvent（onTouchListener→onTouchEvent）。〔册5・五〕

【追问链】→ 事件为什么必须经过 system_server（仲裁/拦截/ANR 计时）？→ 通道为什么是 socketpair 不是 binder（高频小包+fd 复用+不被线程池策略绑架）？→ MOVE 的批量合并（resample）在哪层做（Reader/Dispatcher）？

Q3.2 输入 ANR 的 5 秒从哪一刻开始计?

【深度解答】从 dispatcher publishMotion（写入目标 socket）时刻起，AnrTracker 按截止期排堆；App 消费后 finishInputEvent 回执摘除。5s 未回执→InputManagerService.notifyANR→AMS AnrHelper.appNotResponding（:71，队列化执行 dump）→traces+弹窗。所以输入 ANR=主线程没能在 5s 内把事件"走完责任链并回执"。〔册5・五/六〕

【追问链】→ ANR 弹窗期间事件还投递吗（暂停该窗口投递）？→ 怎么从 traces 定位"谁堵了主线程"（看主线程栈+InputStage 位置）？→ 广播 10s/服务 20s 与输入 5s 的差异逻辑（在场感）？

Q3.3 View 事件分发的四条规则?

【深度解答】①拦截权在父（onInterceptTouchEvent，仅 ViewGroup）；②命中权在子（DOWN 逆 z 序 hit test）；③锁定起于 DOWN（mFirstTouchTarget 定终身，后续 MOVE/UP 不再命中测试）；④消费可中途易主（每轮 MOVE 复议拦截；子可 requestDisallowInterceptTouchEvent 投票但 DOWN 前无效）。滑动冲突官方解（内外拦截/投票）皆由此推导。〔册5・五〕

【追问链】→ ACTION_CANCEL 什么时候来（窗口失焦/被拦），收到必须做什么（清按下态）？→ 多指 split-touch 如何分发（可拆多窗口）？→ onTouchListener 与 onTouchEvent 谁先（前者，return true 短路）？

Q3.4 键盘事件与触摸事件的寻址差异?

【深度解答】触摸按"命中"（可见区域测试，可多目标 split）；按键按"焦点"（InputDispatcher 的 focusedWindow 唯一），路径先过系统拦截器（电源/音量短路）→窗口→ViewRootImpl.processKeyEvent→Ime 截胡→dispatchKeyEvent 自焦点 View 冒泡至 Activity。

两条寻址线在 WMS 汇合（焦点授予随窗口层级）。〔册5・六〕

【追问链】→ 焦点窗口为空时按键去哪（电话/全局策略）？→ 输入法为什么能截胡（ImeInputStage 位置）？→ back 键的手势导航（从区域命中转为 key 合成）？

Q3.5 dumpsys input 能回答哪些问题?

【深度解答】五段活地图：设备清单与配置（触摸屏分辨率/方向）、InputChannel 注册表（哪个窗口哪个 fd）、焦点窗口与触摸目标（寻址现场）、事件队列深度（积压=慢消费）、ANR 计时（哪些事件在等谁）。排障第一站：先看焦点/通道，再看队列，最后下钻 App。〔册5・附录〕

【追问链】→ focusedWindow 为空怎么排查（token/层序/启动态）？→ 队列深但 App 不卡说明什么（回执路径断/monitor）？→ getevent 与 dumpsys input 的结论矛盾时信谁（分层定界）？

Q3.6 为什么 Choreographer 四车道固定 INPUT→ANIMATION→INSETS→TRAVERSAL?

【深度解答】同帧因果闭环：先消化这一帧的输入，再算动画目标值，然后 insets 协商，最后一次性测量布局绘制——顺序颠倒会把输入延迟整整一帧（体感跟手性下降）。常量定义 Choreographer.java:226/256。〔册5・四〕

【追问链】→ doFrame 晚到怎么补偿（跳帧计数+对齐帧间隔）？→ 动画为什么能不阻塞主线程（RenderThread 属性动画）？→ postCallback 的排队单位是什么（车道+VSync）？

Q3.7 同步屏障是什么、为什么 traversal 能插队?

【深度解答】scheduleTraversals（ViewRootImpl.java:2257）postSyncBarrier 让 Looper 队列跳过普通消息、只取异步消息；vsync 到达的 doFrame 消息是异步的，得以越过积压消息立即执行；doTraversal 里 removeSyncBarrier 恢复公平。帧的时效性 > 消息公平性，窗口期极短所以可接受。〔册5・三〕

【追问链】→ 忘拔屏障会怎样（普通消息全部饿死，ANR）？→ Handler 消息能设异步吗（setAsynchronous，系统专用居多）？→ 这与消息优先级（when）是一回事吗（不是，屏障是二分闸）？

Q3.8 三缓冲为什么是三?

【深度解答】双缓冲：App 画一块、SF 合一块，任一方晚点就互等（帧率腰斩）；三缓冲加一块"排队垫"，容忍一次抖动（App 晚一帧用户看到的仍是旧帧而非空等）；四缓冲纯增延迟与内存。BufferQueue 由生产者（App 的 Surface）与消费者（SF/Layer）协商槽位。〔册5・四〕

【追问链】→ 帧在哪个缓冲里被"看见"（SF acquire 后 present）？→ Overlay 直通为什么能绕过 GPU（层不混合）？→ FrameTimeline 怎么标掉帧责任（expected vs actual 四段分责）？

Q3.9 窗口焦点、触摸目标、输入通道三者的关系?

【深度解答】通道是"管道"（addWindow 时配对注册，无通道窗口收不到事件），焦点是"地址"（key 寻址），触摸目标是"收件人"（motion 命中，可 split 多窗口）。三者独立：可见无焦点窗口可收触摸（如悬浮按钮）但收不到按键；有通道无焦点的对话框同屏可点。〔册5・一/六〕

【追问链】→ 点击穿透的机制解释（hit test 未命中本窗）？→ 多屏焦点（per-Display 独立）？→ 无障碍服务会改事件流吗（AccessibilityInputFilter 在责任链内）？

Q3.10 InputReader/Dispatcher/EventHub 三线程为什么不能合并?

【深度解答】三道隔离舱：EventHub 面对设备 IO 抖动（epoll 等待、插拔风暴），Reader 面对翻译复杂度（配置/坐标/手势合成/重采样），Dispatcher 面对策略（焦点/拦截/ANR 计时）。合并后任一环节卡=整条链卡；分离后队列解耦，可观测（各自的队列深度即水位线）。线程边界=可靠性边界。〔册5・五〕

【追问链】→ 触摸重采样（预测补偿）发生在哪层（Reader，相邻事件插值）？→ Dispatcher 卡住的表现（全机输入失灵，dumpsys input 队列爆炸）？→ inotify 热插拔新触摸屏何时可用（Reader 下轮 scan）？

第四章 AMS 与进程管理（12 题）

Q4.1 冷启动十步，进程切换发生在哪几步?

【深度解答】三次跨进程往返：①-②App→system_server（startActivity/ATMS）；④-⑤zygote 孵化（AMS→zygote socket→fork）与 attachApplication（新进程→AMS 回报）；⑥-⑦AMS→新进程（ClientTransaction）。App 侧八步（handleLaunchActivity→performLaunchActivity:3546→onCreate→首帧）全在本进程。观测对照 am start -W 的 TotalTime 与 systrace 轨道分段。〔册4・一/十〕

【追问链】→ 热启动少了哪段（孵化+attach）？→ system_server 视角的"启动完成"信号（activityIdle 回执）？→ 首帧和 TotalTime 谁先谁后（首帧先，idle 后）？

Q4.2 ClientTransaction 事务模型解决什么、怎么执行?

【深度解答】Android 9 前几十个 scheduleXxx 独立 binder 调用乱序风险高；事务化后一次投递"callbacks+终态"（realStartActivityLocked:902 组装），App 侧 TransactionExecutor 先跑 callbacks 再 executeLifecycleState——cycleToPath 沿状态机补中间态（onCreate→RESUME 必经 onStart），路径由执行器推导。批量性、可推理性、可扩展性（新增 Item 即新增周期能力）。〔册4・二〕

【追问链】→ 旋转重建是哪些 Item 的组合（ConfigChange/Relaunch）？→ activityIdle 为什么是"还款日"回执？→ 双向事务（App→AMS 走 ActivityClientController）同一模型吗?

Q4.3 四大组件的"欠条-兑现"分别叫什么?

【深度解答】mPendingServices（Service，bringUpServiceLocked:1179 进程不在先孵化）、mPendingBroadcast（Broadcast，等接收者进程）、mLaunchingProviders（Provider，等宿主发布）、Activity 的 pending 事务队列。统一兑现点：attachApplicationLocked（新进程回报 IApplicationThread 后集中还款）。〔册4・一/三〕

【追问链】→ 为什么集中兑现（收件人就位是共同前提）？ → 兑现顺序有讲究吗（Provider→Broadcast→Service 按语义急迫度）？ → 孵化失败欠条怎么清（进程死亡回调清账）？

Q4.4 有序广播为什么会连锁 ANR？goAsync 救什么？

【深度解答】有序队列串行：前一 receiver 不 finishReceiver 后面全部等待；超时线 10s（前台）/60s（后台，BroadcastQueue.java:203）到点 broadcastTimeoutLocked→AnrHelper。goAsync 把 onReceive 的处理窗口挪到自己线程（PendingResult），但超时线不变——finish 必须限时调。ANR 后广播链跳过该 receiver 继续（单点故障不扩散）。〔册4・四〕

【追问链】→ 并行队列为什么不易 ANR（不等回执）？ → manifest 接收者为什么进有序队列（可截获语义要求串行）？ → 粘性广播的现代替代（onStartCommand/WorkManager 回执）？

Q4.5 ContentProvider 为什么是唯一同步阻塞的组件入口？

【深度解答】调用方语义上要数据（query 返回值），无法欠条化：getContentProviderImpl（ContentProviderHelper.java:158）持 AMS 大锁、宿主不在则孵化并 Object.wait，publishContentProviders 唤醒；兜底 10s 超时。数据面不走 binder 大包：CursorWindow 共享内存分页、openFile 传 fd。〔册4・五〕

【追问链】→ 首次 query 卡顿的常见原因（宿主冷启动 500ms+）？ → 为什么 provider 能抬高宿主进程优先级（incProviderCount:261 计入 computeOomAdj）？ → multiprocess 声明为什么极少用？

Q4.6 computeOomAdj 的因子顺序为什么重要？

【深度解答】核算短路式：从 instrumentation/顶层/可见/前台服务/广播/服务/上应用/缓存 provider 逐项判，**命中即定档**（OomAdjuster.java:421 入口）。顺序=优先级：TOP_APP 永远最前。输出双刻度：PROCESS_STATE 给框架消费（JobScheduler 配额、binder 限制）、oom_score_adj 给 lmkd。〔册4・六〕

【追问链】→ 保活为什么难（自抬优先级的口子都要资源背书）？ → 冻结与杀的先后（先冻结缓冲带，压力升级才 kill）？ → lmkd 怎么知道该杀谁（adj 位图+PSI，LMK_PROCPRIO 命令族）？

Q4.7 Service 三种返回值与复活退避？

【深度解答】START_NOT_STICKY/STICKY/REDELIVER_INTENT 是"复活意愿声明"；ActiveServices 用指数退避（restartDelay 族常量）防杀-启死循环。前台服务=通知背书+优先级抬升；13 的 FGS 通知延迟（离开 10s 后显示部分类型）只是弱化打扰不取消透明。〔册4・三〕

【追问链】→ startForegroundService 的 5s 大限（未调 startForeground 即异常）？ → bind 与 start 两本账（双清才销毁）？ → 后台 startService 熔断（IllegalStateException）怎么绕（FGS/JobScheduler，别绕）？

Q4.8 ANR 的五个触发源与统一出口？

【深度解答】输入 5s（AnrTracker）、前台服务 20s/后台 200s、广播 10s/60s、provider 10s、（+Watchdog 系统级）。统一出口 AnrHelper.appNotResponding（:71）——队列化执行（不在触

发线程做 dump 重活），产出 /data/anr traces+dropbox，弹窗经权限校验。〔册4・四/六〕

【追问链】→ traces 里先看什么（主线程栈+锁持有者）？→ ANR 弹窗为什么另一个进程弹（不信任将死进程）？→ 线上 ANR 归因怎么建（ANR 前最后 binder 事务）？

Q4.9 AMS 与 ATMS/WMS 的分家边界?

【深度解答】2019 拆分：am/ 管进程与组件调度（ProcessRecord），wm/ 管任务/窗口/焦点（ActivityRecord/WindowState）；桥是 WindowProcessController（WMS 持有的进程视图）与 IApplicationThread（AMS 向进程发事务的通道）。窗口可见性回写影响 AMS 优先级——两视图必须联动。〔册4・一/册5・一〕

【追问链】→ 一个 Activity 的两边账本（ActivityRecord↔ActivityClientRecord，token 为桥）？→ 谁决定"后台杀谁"（AMS），谁决定"画到哪"（WMS）？→ 双视图架构的面试一句话（调度与画面解耦）？

Q4.10 进程从 zygote 出生到"可被调度组件"要过几关?

【深度解答】五关：fork 特化（uid/gid/caps/seInfo）→zygoteInit 三步（binder 线程池在 nativeZygoteInit 就位）→attachApplication 报到（ProcessRecord 入库）→集中兑现欠条（四组件）→首次 computeOomAdj 定档。此后进程进入调度生态（adj/cgroup/冻结三件套待遇）。〔册4・一/六〕

【追问链】→ attach 前收到对它的调用怎么办（等/排队，mProcessList 发起方挂起）？→ 进程起来但 Application.onCreate 卡住的后果（一切组件欠条挂着）？→ 预热（空白进程池）的现代做法与限制?

Q4.11 dumpsys activity 能拆出哪四本账?

【深度解答】activities（任务栈/ActivityRecord 状态机）、services（ServiceRecord/绑定/前台）、broadcasts（双队列与超时）、providers（发布表与引用计数）；外加 processes（ProcessRecord/adj/state）——第五本进程账。排障按"症状选账本"：卡启动看 activities，卡后台看 processes+lmkd。〔册4・附录〕

【追问链】→ 一个进程多条 adj 变化的时间线怎么查（events log am_adj）？→ broadcasts 队列里 receiver 卡住叫什么（dispatched 状态超时）？→ providers 引用计数泄漏的表现（宿主杀不掉）?

Q4.12 面试压轴：设计一个"保活"正确打开方式?

【深度解答】正面回答分三层：①合法驻留=前台服务（类型化通知）+ WorkManager 周期任务；②复活链路=STICKY/REDELIVER+JobScheduler 重试+BOOT_COMPLETED 补签；③接受现代现实：12+ 冻结与缓存池容量限制下，"保活"应改为"快速恢复"（进程死亡状态持久化+冷启动优化 <400ms）。把"对抗系统"翻译成"配合系统"才是加分答案。〔册4・六〕

【追问链】→ 冻结期间 FGS 会怎样（解冻窗口期）？→ 厂商白名单机制与 AOSP 差异（后台栈策略厂化）？→ 如何度量"恢复速度"（am start -W 分段+埋点）？

第五章 WMS 与渲染（10 题）

Q5.1 WMS/SF/App 的职责三分离?

【深度解答】WMS 管户口与层位（WindowState/token/z-order，无像素）；App 生产内容（View 树→Surface）；SF 合成上屏（跨进程 BufferQueue）。"系统从不碰你的像素，只调度你的画布"——崩溃域与性能域同时隔离：App 渲染崩不连累系统，SF 卡不拖垮窗口管理。〔册5・二〕

【追问链】→ SurfaceControl 与 Surface 的关系（图层句柄 vs 生产者接口）？→ relayoutWindow(:2219) 交付什么？→ 窗口动画谁驱动（WMS 编排，SF 呈现）？

Q5.2 addWindow 的审验漏斗?

【深度解答】四步从粗到细：屏（DisplayContent 收不收）→token（对得上注册的 WindowToken 吗，BadToken 的故乡）→类型权限（SYSTEM_ALERT_WINDOW 等逐类型）→建籍（WindowState+输入通道配对+层序指派）。顺序即安全语义漏斗，任何一步拒都有专有错误码回流。〔册5・一/二〕

【追问链】→ 对话框 BadToken 的时序解释（Activity attach 前弹）？→ 悬浮窗盖状态栏需要什么（type 层级+权限，非"画得高"）？→ 子窗口的 token 挂谁（父窗口）？

Q5.3 performTraversals 三幕的规矩与禁忌?

【深度解答】measure（自顶向下发 MeasureSpec，wrap_content 常两轮）→layout（拍板位置，期间再 requestLayout 即抛异常防震荡）→draw（软件逐条/硬件录 DisplayList 由 RenderThread 执行）。三幕只发生在 ViewRootImpl 一处；invalidate 是欠条（同帧合并），requestLayout 不触发 draw。〔册5・三〕

【追问链】→ onCreate 里 getWidth()==0 的原因（解析≠演算）？→ 为什么 UI 单线程（checkThread 免锁代价）？→ 硬件加速局部重绘为什么高效（子树重录整树重放）？

Q5.4 z 序三定律?

【深度解答】type 定车道（应用 1~99/子窗 1000+/系统 2000+）；Task 序定同车道名次（前台任务整体靠上）；动画期间暂存层例外（动画器接管结束归还）。z 序=layer（type 基线）+同层序+动画暂存，三句话解释一切"谁盖谁"问题。〔册5・一〕

【追问链】→ 输入法窗口的层位策略（perFocus 动态）；→ PopupWindow 为什么是子窗口（随父同进退）；→ 分屏的层位（Task 分区独立基线）？

Q5.5 Insets 是裁剪还是协商?

【深度解答】协商：InsetsController 与 WMS 按 InsetsType 分通道谈（status/nav/ime 独立），behavior 声明姿态；键盘动画两侧同帧同步（InsetsAnimationThread 免被主线程拖累）。adjustResize=Insets 全量应用重测；adjustPan=整窗平移不重测。键盘是独立窗口，只是协商出的量进入你的测量参数。〔册5・一〕

【追问链】→ edge-to-edge 的现代做法（WindowInsets API 自理）；→ 键盘"顶起布局"的测量链（insets→MeasureSpec）；→ 刘海屏（displayCutout）与 insets 的关系（另一通道）？

Q5.6 Choreographer 的两个实例与双时钟?

【深度解答】UI 线程 getInstance() 编排测量布局；RenderThread 的 getSfInstance()（ViewRootImpl.java:925 引用）服务硬件渲染提交——两个时钟域对应"主线程画什么、渲染线程怎么画"。App 的 VSync 与 SF 的 VSync 相位错开（生产/合成流水化）。〔册5・四〕

【追问链】→ 谁给 Choreographer 发 VSync（DisplayEventReceiver/BitTube，源自 SF 分发）；→ doFrame 晚到的补偿（跳帧计数对齐）；→ postCallback 与 postDelayed 的车道语义?

Q5.7 硬件加速的一帧后半程?

【深度解答】draw 录 DisplayList（RecordingCanvas）→RenderThread syncAndDrawFrame（Skia GPU 后端执行）→纹理/RenderNode 属性直接改不重录（translationX 流畅的秘密）→queueBuffer 入 BufferQueue（三缓冲）→SF 下个 VSync 合成（Overlay 直通优先）→present。掉帧责任由 FrameTimeline 四段标注（App/RenderThread/SF/Disp）。〔册5・八〕

【追问链】→ 为什么属性动画不 invalidate（改 RenderNode 矩阵）；→ Overlay 命中条件（不混合/无缩放/格式匹配）；→ 强制 GPU 渲染开关改变了什么（软件层也走 GPU）？

Q5.8 窗口不显示的排查三步?

【深度解答】①dumpsys window windows 查该窗：mViewVisibility/isReadyForDisplay/SurfaceControl 是否创建——停在 addWindow（无 relayout）还是首帧未提交；②查层位与 token（被盖/未挂树）；③perfetto 看 traversal 是否发生。三步分别对应"存在了吗/排对了吗/画了吗"。〔册5・附录〕

【追问链】→ Surface 拿到但黑屏（无内容提交，首帧未 queue）；→ 动画后窗口消失（暂存层归还失败/visibility 错）；→ 无障碍服务报告的"可显示"与实际不符（accessibility 视图与窗口树差异）？

Q5.9 多屏/折叠屏的窗口世界?

【深度解答】每屏一个 DisplayContent（窗口树/Insets/焦点独立 per-display）；Task 可跨屏迁移（拖拽分屏=Task 重组而非窗口搬运）；折叠态变化=配置变更广播（重建事务，册4 §2.2 同款）。焦点 per-display 但输入 dispatcher 全局只有一个 focusedWindow per display 的集合。〔册5・六〕

【追问链】→ 分屏两个 App 的输入怎么分（per-task 焦点+命中）；→ 外接屏默认显示谁（uniqueId/displayId 路由）；→ 投屏镜像（VirtualDisplay+镜像 Layer）？

Q5.10 ViewRootImpl 为什么是"窗口总管"?

【深度解答】它一身五职：View 树的根（assignParent 冒泡终点）、窗口 binder 通道（Session）、输入接收者（InputEventReceiver+InputStage）、帧编排者（Choreographer 客户）、Surface 持有者（生产者接口）。Activity/PhoneWindow/DecorView 只是结构，ViewRootImpl 才

把结构接入系统——所以 UI 的所有"与系统打交道"都经它。〔册5・二/三〕

【追问链】→ 一个进程几个 ViewRootImpl（每窗口一个，WindowManagerGlobal 登记）；→ checkThread 在哪类调用触发（traversal 族）；→ SurfaceView 为什么能独立刷新（独立 Surface 打洞，与窗口 Surface 并行提交）？

第六章 ROM / BSP / 刷机（12 题）

Q6.1 BSP 五件套是什么?

【深度解答】板级支持包=①内核+dtb/dtbo（本树 device/google/*-kernel 的 Image.lz4/dtbo.img）②vendor HAL 与其 rc/init 脚本 ③vendor sepolicy ④BoardConfig/Product 产品配置（device/oem/product）⑤固件（modem/tee/bluetooth…）。判断一个 ROM"支持一块新板"就是五件套齐且接口对齐 VINTF。〔notes・07/册1・三〕

【追问链】→ 板卡 first boot 卡在哪查（dtb 不匹配→kernel panic/找不到 console）；→ 新 HAL 该放哪个分区（vendor，接口在 hardware/interfaces）；→ sepolicy vendor 与 system 怎么协同（public/private 划分）？

Q6.2 device/ 目录在构建里的角色?

【深度解答】device/<oem>/<product> 是"产品身份证"：AndroidProducts.mk 列产品名、BoardConfig.mk 定板级（分区尺寸/架构/内核预编译路径）、device.mk 选模块与 overlays。Soong 由此展开变体（书1 §7.2）——lunch 的目标名就来自这里。本树含 amlogic/linaro/google 等多家参考。〔notes・07〕

【追问链】→ 新建产品的最小集（AndroidProducts+BoardConfig+device.mk）；→ PRODUCT_ 变量与 BOARD_ 变量的分工（产品 vs 板）；→ overlay（RRO）怎么改默认值不碰框架？

Q6.3 fastboot 与 fastbootd 的区别?

【深度解答】fastboot：bootloader 里的协议栈（system/core/fastboot/main.cpp:31），可刷 boot/system 等物理分区、解锁；fastbootd：userspace fastboot（走 adbd 通道），能刷动态分区（super 内的逻辑分区）——因为 bootloader 看不懂 super 里的 metadata，只有跑起来的系统（或 recovery）能。A/B 时代逻辑分区的刷机必经 fastbootd。〔notes・07〕

【追问链】→ fastboot reboot-bootloader 怎么实现（BCB/reboot reason）；→ 解锁会清 data 吗（会，防绕过加密）；→ 刷 GSI 的标准流程（解锁→fastboot init boot...→fastboot reboot fastboot→flash system）？

Q6.4 AVB 校验链与四色屏?

【深度解答】vbmeta 是根：内含各分区哈希（或 hashtree 描述符）+回滚索引+公钥；bootloader 逐级验签（boot→vbmeta→system…）。结果四色：绿（锁定全过）、黄（锁定但自定

义根）、橙（解锁）、红（校验失败拒启）。运行时 dm-verity 续验 system 只读分区。〔册1・七/notes・07〕

【追问链】→ avbtool 在构建里哪一步调用（build/make/core/Makefile:1054/4932）；→ 回滚索引防什么（降级攻击）；→ 锁定设备刷第三方 magisk 为何黄屏（boot 被改，自定义验签键路径）？

Q6.5 OTA 的一生（A/B 全流程）？

【深度解答】update_engine（system/update_engine）从源（OTA 包/serving）下载→校验 payload 签名→后台槽写入（未激活槽）→postinstall（动态分区 COW 快照若虚拟 A/B）→boot_control 标记新槽可启动→重启进新槽→成功后 slot 标 active、旧槽可复用；连续失败自动回退旧槽。Payload 的 manifest/metadata 与 delta（差分包）是核心数据结构。〔notes・07〕

【追问链】→ 虚拟 A/B 怎么省空间（COW，写时复制到 data）；→ A/B 的代价（存储双份 boot 分区）；→ 静默升级（seamless）的用户感知边界（重启那一刻）？

Q6.6 recovery 模式还剩什么职责?

【深度解答】A/B 后 recovery 大幅瘦身：sideload（adb 推包，bootable/recovery）、factory reset、部分厂商救援模式（进入内核小环境而非完整系统）。传统"卡刷"已非主线（A/B 无 recovery 分区刷写语义，或仅保留薄壳）。recovery_ui 与 minui 是它的 UI 栈。〔notes・07〕

【追问链】→ 进入 recovery 的路径（BCB boot-recovery/fastboot；）；→ sideload 与 update_engine 的关系（互斥的两种投递）；→ 双清清的是哪两层（data+cache，userdata 与回滚索引）？

Q6.7 刷机镜像家族速记?

【深度解答】boot（kernel[+GKI]）、init_boot（通用 ramdisk，13 起）、vendor_boot（vendor ramdisk）、dtb/dtbo（设备树与覆盖）、vbmeta（AVB 根）、system/vendor/product/odm/system_ext（只读主分区，super 动态切分）、userdata、super（动态分区容器）。产物规则在 build/make/core/Makefile（:846 dtb.img、:985 --dtb 参数）。〔notes・07〕

【追问链】→ GSI 刷哪个（system.img，CTS 认证设备必须可刷）；→ odm 与 vendor 的分工（板级 vs SoC 级）；→ super.img 怎么改子分区大小（lpunpack/lpmake 重建）？

Q6.8 新板卡 first bring-up 的五步?

【深度解答】①kernel 能起（dtb/console/rootfs）②init 进二阶段（分区表/挂载对）③关键服务活（servicemanager/SF/logd 的 rc 与 sepolicy）④显示与输入通（GPU/触摸 HAL+dtbo）⑤过 VINTF 自检（manifest/matrix 对齐）。每步都有"标志性日志"可查——bring-up 的方法论就是"用日志换进度"。〔册2 全册/notes・07〕

【追问链】→ 卡在 kernel 的常见因（dtb 不匹配/console 串口错）；→ 卡在 init 二阶段（fstab/avb/selinux 标签）；→ SF 起但黑屏（HAL/overlay/drm 少一层）？

Q6.9 ROM 定制的"正确姿势"四件套?

【深度解答】①预装应用（system/product 分区+privileged 白名单，册6 §五续）②默认行为（RRO overlay 改资源不动框架）③默认设置（SettingsProvider defaults/vendor 配置）④开机向导（SetupWizard 产品件）。原则：**能在 product/overlay 层解决就不碰 system/ 源码**——否则每轮 AOSP 合并都是灾难。〔册6/notes · 07〕

【追问链】→ 为什么不推荐直接改 framework 源码（升级合并冲突）；→ overlay 能改什么不能改什么（资源值可、代码逻辑不可）；→ 多产品共存的工程组织（device 树继承 lunch 链）?

Q6.10 dm-verity 与数据完整性?

【深度解答】只读分区（system/vendor…）挂载时建 hashtree（merkle），访问块时内核校验（第一块失败→重启/降级 per 模式）；vbmeta 存树根。检测到篡改（root 后改 system）即拒绝读取——配合 AVB 构成"启动前验根、运行中验块"。〔册1 · 七〕

【追问链】→ verity 与 AVB 的分工（启动信任根 vs 运行时块校验）；→ fec（前向纠错）干什么（小损坏自愈）；→ userdebug 为什么能关（disable-verity 调试豁免）?

Q6.11 Cuttlefish 是什么、为什么系统开发离不开它?

【深度解答】本树自带的虚拟设备（device/google/cuttlefish）：以用户态进程模拟手机（crosvm 虚机+虚拟传感器/触摸/显示），宿主机直接跑 AOSP 全栈，支持 adb/fastboot/streaming。价值：免真机做系统实验、CI 全量验证、GSI/内核矩阵测试。系统开发者的"本地沙箱"标准答案。〔notes · 07〕

【追问链】→ 它的 HAL 怎么虚拟（ranchu/cuttlefish HAL 实现）；→ 性能与真机差在哪（GPU virgl/gfxstream）；→ 与模拟器（ qemu arm）的区别（跑真 AOSP 产品构建）?

Q6.12 刷机/系统开发的常用命令拓扑?

【深度解答】四层记忆：宿主工具（fastboot flash boot boot.img / adb reboot bootloader|fastboot|recovery）；设备查询（getprop ro.* / dumpsys -l / service list）；系统操作（pm/svc/cmd activity|window|input|surfaceflinger）；观测（logcat -b events|crash、dumpsys window/input/package、perfetto）。每条命令对应前六册的一个子系统——命令即索引。〔各册附录〕

【追问链】→ cmd activity 与 am 的关系（同一服务的两个门）；→ 无 root 抓开机日志（bootchart/events log）；→ 串口日志与 logcat 的层次（console=dmesg vs 框架）?

第七章 综合场景题（8 题）

Q7.1 场景：用户点击图标 500ms 后才见首帧，怎么定位?

【深度解答】分段定界：am start -W 三段（WaitTime/TotalTime）+systrace：①孵化段（am_proc_start→attach）异常长？查 zygote 忙碌/USAP/进程池；②事务段（realStart→onCreate）

短则正常；③onCreate→首帧（inflate/traversal/GPU）业务大头。按三段归因，各自有专属轨道与锚点（册4 §八实战流程）。〔册4 · 八/册5 · 四〕

【追问链】→ 三段各占多少算正常（孵化 100-300ms/事务 10ms 级/首帧业务定）；→ 若 TotalTime 正常但用户仍觉慢（感知层：动画启动延迟/图标点击反馈）；→ 线上无 trace 怎么近似（埋点分段上报）？

Q7.2 场景：系统升级后某厂商 App 崩溃率暴涨，系统侧还是应用侧?

【深度解答】系统侧排查序：①平台行为变更（targetSdk 分水岭 M/Q/R 的权限与后台限制）②接口兼容（非 SDK 灰名单收紧，veridex 检测）③组件策略（FGS/通知/广播限流收紧）④时机变化（调度更激进致首启动竞态）。若崩溃栈在框架 API 边界但触发条件是"进程被杀重建"，多半是冻结/回收策略——系统"没错"，是契约边界移动了。〔册4 · 六/册6 · 四〕

【追问链】→ 如何复现（冻结窗口期+低内存模拟）；→ 应用侧正确的抗性设计（状态持久化/幂等重建）；→ 报告给 Google/厂商的正确姿势（bugreport+veridex 报告）？

Q7.3 场景：整机偶发输入全失灵 3 秒自愈?

【深度解答】层次定界：getevent 有流→内核正常；dumpsys input 的 dispatcher 队列暴涨→投递端堵（焦点窗口的 App 消费不动或 ANR 计时中）；队列正常但 App 无响应→通道/焦点瞬变（窗口切换动画期）。3 秒自愈指向"某窗口短暂僵死+超时回执/ANR 逃逸（dialog 出现）"——抓现场：连续 debuggerd -b 抓目标窗口栈。〔册5 · 五〕

【追问链】→ 为什么不是输入法劫持（Ime 截胡不阻塞其他窗口）；→ 全失灵与单 App 失灵的分界（focusedWindow 是否换人）；→ ANR 弹窗出现=输入恢复的机制（暂停投递+用户决策）？

Q7.4 场景：设计一个系统级"应用双开"?

【深度解答】三条路线对比：多用户（正统：PackageUserState per-user+userId 隔离，成本是 userId 资源与切换）；克隆应用（应用侧改包名多实例，破签名共享）；虚拟化容器（VirtAPP 类，framework hook，兼容性差）。系统工程师的正解是多用户基建+隐形 profile（工作资料机制复用）：AMS/PMS/WMS 天然 per-user，需要补的是 UI 聚合与通知路由。〔册6 · 五/册4 · 一〕

【追问链】→ 双开的存储隔离边界（per-user data 目录）；→ 通知如何区分（userId 附着 +SystemUI 分组）；→ 与厂商"应用分身"差异（多为方案一的白牌化）？

Q7.5 场景：车载/平板需要"分屏+多任务"，系统侧要动哪些层?

【深度解答】WMS 层：TaskDisplayArea 分区策略（WindowContainer 分派规则）；ATMS 层：分屏模式与 Task 重组（rootTask 复用）；输入层：per-task 焦点与命中；渲染层：双 Surface 独立管线；策略层：多窗口开关（config supportsMultiWindow）与厂商定制度量。AOSP 已有分屏基建（12+），产品化难点在交互策略而非框架能力。〔册5 · 一/六〕

【追问链】→ 两 App 同帧渲染的性能预算（GPU 与 SF 合成压力）；→ 拖拽分屏期间的窗口归属（Task 迁移动画）；→ 分屏下输入法给谁（焦点 task）？

Q7.6 场景：把 Android 13 移植到一块新开发板（bring-up 计划）?

【深度解答】六周节奏模板：W1 kernel/dtb 起 init；W2 分区+sepolicy 跑通二阶段；W3 显示输入通（SF/触摸）；W4 PMS 扫描全量包+开机到 Launcher；W5 VINTF/CTS 基线过；W6 性能与稳定性（bootchart、monkey）。每步验收物=日志证据；风险最大处=GKI 内核匹配与 HAL 版本（VINTF 对不齐全盘皆输）。〔notes · 07/册2〕

【追问链】→ 先 bring-up display 还是 touch（display，可 logcat 盲调）；→ VINTF 对不齐的现象（服务起不来/CTS fail 明细）；→ 何时引入厂商 BSP 包（W2 后，逐 HAL 合入）？

Q7.7 场景：面试官问"读系统源码你最大的收获？"

【深度解答】高分模板=具体锚点+方法论：例——"读 Binder 十跳后，我对'身份在内核采集'的安全模型有了肌肉记忆，之后审任何 IPC 方案先问凭证谁填；读 init 单线程模型后，我把'确定性优先于吞吐'用在了一个调度器设计上。"要点：**一个让你改变设计观的具体机制**，而不是"我读了多少行"。〔各册 · 设计决策章〕

【追问链】→ 这套源码观怎么迁移到非 Android 系统（分层/身份/事务化是通用的）；→ 六册里最推荐新人先读哪册（第一册+第二册：地图与主线）；→ 保持源码新鲜的习惯（release notes+Gerrit 关注几个核心仓库）？

Q7.8 场景：给你一台卡顿设备做 30 分钟体检?

【深度解答】五步流水：①定界：dumpsys gfxinfo/framestats 或 perfetto 抓 10s（掉帧责任段）；②系统水位：dumpsys meminfo+lmkd 日志（是否内存挤压杀进程循环）；③调度：top/dumpsys activity processes（adj 错乱/缓存风暴）；④单点深挖：FrameTimeline 红帧下钻对应册 · 章（App traversal? SF 合成？）；⑤复验：改一项再测。30 分钟的本质是"用流水线代替直觉"。〔册5 · 四/八、册4 · 六〕

【追问链】→ 优先怀疑系统还是应用（数据说话：FrameTimeline 分责）；→ 内存压力型卡顿的特征（am_kill+am_proc_start 频繁成对）；→ 给用户的一句话结论怎么写（现象+责任段+建议三段式）？

第八章 深水区题（20 题）

Q8.1 ART 的并发复制 GC 为什么能做到毫秒级停顿?

【深度解答】三件套：region 化堆（RegionSpace，搬移以区域为单位）+ **读屏障**（引用读取时顺带检查 forwarding 指针、按需搬移——搬移与应用并发）+ 卡表/记忆集（跨代引用脏页索引，避免全堆扫描）。STW 只剩根初始标记与最终修正两个小阶段。代价：每条引用读指令多几条屏障指令——用 CPU 换延迟。锚点 art/runtime/gc/collector/concurrent_copying.cc。〔册4 · 七〕

【追问链】→ LOS 大对象为什么独立空间? → GC 与 zygote COW 的互动（fork 前 gcAndFinalize 的意义）？ → 观测 GC 的三件工具（meminfo/systrace gc 轨/heapdump）？

Q8.2 MTERP 为什么用汇编写解释器?

【深度解答】C 解释器每条字节码都有函数调用/栈帧开销；MTERP（art/runtime/interpreter/mterp/arm64ng）把"取指-分发-执行"编成一段手写汇编：分发表跳转、虚拟寄存器钉在物理寄存器、线程切换点内联——吞吐近翻倍。它不是 JIT：仍是一条条解释，只是"解释的地板"被垫高。〔册4・七〕

【追问链】→ MTERP 与 quickened 字节码的关系？→ 为什么不全部 JIT（编译也有成本，冷代码白编）？→ profile 如何决定谁进 JIT 谁进 AOT?

Q8.3 lmkd 的 PSI 事件是什么、比轮询好在哪?

【深度解答】PSI（Pressure Stall Information）是内核度量的"因内存压力而停顿的任务时间占比"；lmkd（lmkd.cpp:3735）订阅 some/full 两档阈值事件，压力到达即刻按 adj 位图选杀——**事件驱动、零轮询延迟**，且能区分"有富余但抖动"与"真饥荒"。旧内核态 LMK 只看剩余水位，粒度粗且策略不可更新。〔册4・七〕

【追问链】→ LMK_PROCPRIO 谁在发（AMS OomAdjuster）？→ zram writeback 的触发条件？→ 为什么杀了还卡（swap thrash， PSI full 档兜底）？

Q8.4 冻结（freezer）与杀（kill）的配合顺序?

【深度解答】CachedAppOptimizer 先冻结（cgroup v2 freezer，CPU 全停+BR_FROZEN_REPLY 快速失败+广播 deferred），内存仍紧才升级 kill——冻结是可逆档（解冻省一次冷启动），kill 是不可逆档。观测点：dumpsys activity 里 cachedFreezer 状态与 logcat 的 freeze/unfreeze 事件。〔册4・七/册3・十〕

【追问链】→ 冻结进程的 binder 调用方如何兜底（重试/退避）？→ 解冻窗口谁开（AMS 在调度组件前）？→ 冻结与 JobScheduler 的互斥怎么处理?

Q8.5 cgroups.json 与 task_profiles.json 的分工?

【深度解答】cgroups.json 定义**层级挂载**（哪些控制器、挂哪、属主）；task_profiles.json 定义**命名任务档**（如 ProcessCapacityHigh/SchedUgTop=一组 cgroup 写入+调度参数），服务 rc 与 framework 代码引用档名而非底层路径——**单一事实源+可移植**（Android 版本/厂商差异由 profiles 吸收）。〔册4・七〕

【追问链】→ 新建一个自定义档的步骤？→ cpuset 与 uclamp 各管什么（绑核 vs 提升裕量）？→ init.rc 的 task_profiles 行何时兑现（Service::Start）？

Q8.6 传感器为什么用 FMQ 而不是 Binder?

【深度解答】传感器事件高频小包（200Hz+ 的 9 轴流），binder 每次事务有组包/调度成本；AIDL HAL 的 EventQueue（ISensors.aidl:129）用 **FMQ（共享内存环形队列）+ 事件旗标**：写者（HAL）入队、读者（SensorService）批量收割，控制面（启停/配置）才走 binder——数据面/控制面分离的传感器版。〔册5・八〕

【追问链】→ direct channel 比 FMQ 更快在哪（连 SensorService 都绕过，DSP 直写共享内存）？→ 批处理（batch）的 FIFO 硬件前提？→ 融合传感器（CorrectedGyro 等）在哪算（service 侧

Fusion.cpp）？

Q8.7 电源键在哪儿被"没收"的？

【深度解答】两道闸：入队前 interceptKeyBeforeQueueing（InputDispatcher.cpp:3935→InputManagerService.java:3003→WMS InputPolicy）与分发前 interceptKeyBeforeDispatching；电源/音量组合（截图、复位）在系统策略层直接消费并标记"不再分发"——App 永远收不到原始事件。这也是防键盘记录类攻击的机制层。〔册5・八〕

【追问链】→ App 能拦截音量键吗（能，除非系统已消费）？ → 按键与触摸的"没收权"差异？ → 无障碍为什么能读到被拦按键（a11y 通道在拦截上游）？

Q8.8 预测性返回改变了什么？

【深度解答】旧模型：返回=KEYCODE_BACK 到达才响应，用户无预览；新模型（OnBackInvokedDispatcher.java:28）：手势按下即向目标窗口发渐进回调（progress/start/cancel），App 可实时预览并取消，系统侧同播手势动画——**从"事件通知"进化为"过程协商"**。迁移期两套并存，按 targetSdk/opt-in 切换。〔册5・八〕

【追问链】→ 与手势导航的手势动画如何协同？ → WebView 内部返回谁优先？ → 未适配 App 的降级行为（自动回退旧 KeyEvent）？

Q8.9 IME 卡顿常见根源？

【深度解答】文字协议滥用：InputConnection 每次查询都是一次 binder 往返（getTextBeforeCursor/setComposingText…），输入法每键多次查询即雪崩；IMMS 侧重绑（InputMethodBindingController）与焦点切换竞态也会表现为"键盘冻住"。排查：systrace 的 IME 轨+文本协议调用计数；优化：批量编辑（BatchEdit）、减少上下文查询。〔册5・八〕

【追问链】→ 键盘弹出的 Insets 协商链？ → 多输入法切换的绑定生命周期？ → 手写就地输入（13）绕开了什么（不走文本框、Surface 直接递笔迹）？

Q8.10 无障碍服务为什么被严格管控？

【深度解答】a11y 服务可读全屏内容+合成触摸/按键（AccessibilityManagerService 的事件流与 GestureDescription 注入）——等同"上帝权限"。13 起强制安装来源受限+显式授权弹窗、限制后台绑定。设计启示：能力越普惠（残障辅助）越要防能力滥用（羊毛/监控）。〔册5・八〕

【追问链】→ AccessibilityNodeInfo 快照的生成成本（View 树按需序列化）？ → 触控朗读与 TalkBack 的事件消费差异？ → 自动化测试（UiAutomator）用的是不是同一条通道（是）？

Q8.11 neverallow 在 SELinux 里是什么地位？

【深度解答】策略"宪法条款"：编译期强制检查，任何 allow 与既有 neverallow（如 system/sepolicy/private/domain.te:119/:137 起的数百条）冲突即构建失败——**把"永远禁止"从 review 纪律升级为机器保证**。写策略的正确姿势：先查 neverallow 是否已封死此路，再谈 allow。〔册1・十六〕

【追问链】→ audit2allow 为什么不能无脑用？→ seapp_contexts 给谁打标（App 进程域，按包名/uid）？→ MLS 类别（s0）在实际策略里怎么用？

Q8.12 APEX 回滚的两次启动判定是什么？

【深度解答】staged 会话激活后处于"试运行"：boot 阶段 apexd/_bootflag 标记，若本会话期间系统连续两次未正常 boot 到 reckoned 状态，下次启动自动回退旧版 APEX 并禁新会话冷却——与会话状态机（staged→activated→reckoned）一体。这是 Mainline 敢高频发版的"保命绳"。〔册 1・十六〕

【追问链】→ 回滚与 APK 的 rollback（RollbackManager）什么关系（不同子系统、同一思想）？→ 哪些 APEX 不允许回滚（与 boot 链绑定的）？→ 如何观测（apexd --list/dm 命令族）？

Q8.13 链接器命名空间解决了什么 Treble 痛点？

【深度解答】动态符号解析的隔离：framework 进程看不到 vendor 命名空间私有导出、vendor 进程用的是 vndbinder/libc 的 vendor 变体——**同名 so 双版本并存不互踩**（system/linkerconfig 生成各进程 linker config）。它发生在链接期，比运行期任何隔离都早。〔册 1・十七〕

【追问链】→ "dlopen failed: library not found"的命名空间排查法（readlink /proc/<pid>/maps + linker config 对比）？→ sphal/llndk/default 四类命名空间各放什么？→ NDK app 链的是哪个域（default，白名单 public libs）？

Q8.14 fastboot 为什么要有 userspace 版（fastbootd）？

【深度解答】动态分区（super）的元数据只有完整系统里的 liblp/dm-linear 能解析；bootloader 无此栈。逻辑分区的 flash 因此必须走 fastbootd（userspace，经 adbd 通道）；物理分区（boot/vbmeta…）才由 bootloader 版直刷。刷 GSI 流程"reboot fastboot"就是为了进入这个 userspace 环境。〔册2・十一〕

【追问链】→ A/B 槽后缀与 slot 切换谁管（boot_control HAL）？→ getvar:max-download-size 决定什么（分块下载尺寸）？→ 解锁状态影响哪些校验（bootloader 层 AVB，dm-verity 运行层仍提示）？

Q8.15 update_engine 的 Action 流水线？

【深度解答】InstallPlan→FilesystemVerifier（验旧）→Download/DeltaPerformer（增量块搬运+bz4→写后台槽）→Postinstall→boot_control 切槽。任一 Action 失败整单回退、旧槽无损；payload 签名与哈希树两级校验。观测：logcat -s update_engine 与 `update_engine_client --status`。〔册2・十一〕

【追问链】→ 增量包怎么对齐新旧块（manifest 的 install operations）？→ 虚拟 A/B 的 COW 写在哪（/data，牺牲空间换速度）？→ postinstall 脚本能做什么（OTA 后 dexopt 触发）？

Q8.16 GKI 把什么从 boot 分区拿走了？

【深度解答】通用内核镜像：boot 只剩 Google 签名内核；厂商差异（模块/vendor ramstock→vendor_boot、dtbo、bootconfig 参数）外迁，内核与 vendor 由符号清单（KMI）锁界面——内核碎片化治理。bootconfig 是 cmdline 的结构化继任（androidboot.* 家族迁入）。〔册2・十一〕

【追问链】→ KMI 符号清单怎么维护（abi_gki_*.xml 白名单）？→ vendor_dlkm 模块谁加载（内核 module loader+init rc）？→ GKI 与 Project Treble 的关系（镜像版同一思想）？

Q8.17 installd 的缓存清退什么时候触发?

【深度解答】磁盘水位紧张时系统存储策略（StorageManagerService）调用 installd 的 rm_cache 系列：CacheTracker 按 app 聚合 cache 用量、先清"最久未用+无活跃进程"的份额，必要时清至配额；App 无感知（cache 语义本就允许蒸发）。排查"cache 变小/quota 异常"第一站 dumpsys diskstats + installd 日志。〔册6・八〕

【追问链】→ cache 与 data 的物理区分（目录位置+清退权差异）？→ 配额的底层实现（fs quota 或 xattr）？→ CrateManager 的 crate 是什么（scoped storage 的分箱目录）？

Q8.18 Auto Backup 与 Key/Value Backup 的边界?

【深度解答】Auto Backup：全量 tar 流（adb backup 同源），25MB 上限，应用零代码；K/V：BackupAgent 逐键增量、需写代码，适合状态型数据。共同基建：BMS（services/backup）调度、传输层可插拔（Google/OEM）、恢复发生在安装流水线"校验后首启前"窗口。工作资料独立密钥域。〔册6・八〕

【追问链】→ backup 过期标记谁打（安装会话后，BMS）？→ 恢复失败会阻断首启吗（不阻断，只标记待重试）？→ D2D 迁移走哪条通道（近场直传，不走云）？

Q8.19 split APK 的加载与一致性?

【深度解答】系统侧一个 Package 多 APK：安装会话一次提交（签名全一致）；运行时 LoadedApk 建 split classloader 路由资源与 dex（base 与 config.split_xx 按需组合）；Feature Delivery 的按需安装只是会话化在模块粒度的复用。风险点：split 间资源覆盖顺序、跨 split 反射（同签名同 visibility 才稳）。〔册6・八/册4・一〕

【追问链】→ config split（mcc/mnc/abi/density）谁挑（最佳匹配算法在 LoadedApk/AssetManager）？→ 按需模块卸载的边界（保留 base 依赖闭包）？→ Play Install 与 pm install-session 的关系（同一会话 API）？

Q8.20 RPC backend 会取代 Binder 吗?

【深度解答】13 树里 RpcState.cpp 标志的 binder_rpc 实验分支：Binder 协议跑在 Unix socket/TLS 等任意传输上，面向多设备（手机-车机-穿戴）——**Binder 正在从"设备内 IPC"升维为"跨设备 RPC"**。但设备内主线不变：/dev/binder 的驱动优势（一次拷贝/内核身份）无法被用户态传输复制。答案是共存：域内 Binder、域间 RPC。〔册3・十二〕

【追问链】→ RPC 后端如何做安全（TLS+会话密钥，替代驱动身份）？→ 与跨设备"超级终端"框架的关系？→ AIDL 四后端（java/cpp/ndk/rust）在 RPC 下的复用？

第九章 补全题（10 题）

Q9.1 主线程死循环为什么"不卡"?

【深度解答】"卡"的前提是循环在忙转；而 Looper.loop（Looper.java:260）的 next() 无消息时经 MessageQueue.nativePollOnce（:72/:335）进入 native：NativeMessageQueue（android_os_MessageQueue.cpp:41/107）→ libutils Looper.pollOnce（Looper.cpp:181）→ **epoll_wait（:245）睡眠**，线程挂起零 CPU；nativeWake（MQ:73/:443）写管道唤醒。主线程=一台"用 epoll 武装的优先级队列"，idle 成本近乎为零。〔册4 · 八〕

【追问链】→ 同步屏障如何插队（异步消息优先，书5 §3.1）？→ epoll 还监听谁（输入 socket/IdleHandler fd——书5 输入回调的底座）？→ ANR 时这台发动机怎么了（主线程在处理某条消息/睡眠但队列被屏障压住）？

Q9.2 post(Runnable) 和 sendMessage 的区别?

【深度解答】殊途同归：post 把 Runnable 装进 message.callback，dispatchMessage（Handler.java:97）第一优先级先跑 callback，其次 mCallback，最后 handleMessage——**API 差异只是回调挂载点不同**。时序上都要走同一队列同一 when 排序；跨线程 post 即"投递到目标 Looper"的标准姿势（binder 线程转主线程即此，书4 §C.3）。〔册4 · 八〕

【追问链】→ handleMessage 里再 sendMessage 会怎样（正常入队，潜在自旋需警惕）？→ IdleHandler 何时跑（next() 队列空的空档）？→ 消息池复用（obtain/splice 回链表）为什么重要（免 GC 抖动）？

Q9.3 JNI 三类引用各管什么?

【深度解答】局部引用：方法帧内生命周期（循环里长期持有需显式 DeleteLocalRef——表有上限）；全局引用：跨线程/长存（NewGlobalRef/DeleteGlobalRef 成对，BinderProxy 的 nativeData 即现场）；弱全局：不保活（promote 失败即对象已亡，JavaBBinderHolder 缓存即现场）。异常规则：JNI 异常不清空、native 继续跑，必须 ExceptionCheck 早退。〔册1 · 十八〕

【追问链】→ 静态注册 vs 动态注册（RegisterNatives vs JNI_OnLoad）？→ 框架为什么全静态（启动一次性注册大表，可校验签名）？→ native 线程怎么 attach（AttachCurrentThread，detach 泄漏线程局部表）？

Q9.4 为什么 Application Context 启动 Activity 会报错?

【深度解答】隐式约束：startActivity 需要窗口令牌（token），而令牌来自 Activity/窗口体系——Application 的 ContextImpl（ContextImpl.java:192）没有绑定窗口。例外：加 FLAG_ACTIVITY_NEW_TASK 可以（走任务栈而非"在调用者窗口上叠"路径）。本质是 Context 族谱的权限投影：**Context 是"环境句柄"，能干什么取决于它背后的实体**。〔册1 · 十八〕

【追问链】→ Service 的 Context 有令牌吗（没有 UI 令牌，同限制）？→ 系统侧"系统 Context"（createSystemContext）从哪来？→ getSystemService 的缓存挂在谁身上（ContextImpl，per-context）？

Q9.5 sp/wp 与 Binder 引用计数命令什么关系？

【深度解答】sp<T>/wp<T>（RefBase.h:281）是进程内的强/弱计数；BC_ACQUIRE/BC_INCREFS（书3 §5.4）把它**跨进程镜像**：本端 sp 构造→发 ACQUIRE→驱动在对端建引用记账，两端各自确定性析构。所以 Binder 的"对象生命周期"= RefBase 本地 + 驱动远程账本的对偶；wp.promote 失败≈对端已死（死亡通知兜底）。〔册1・十八/册3・三〕

【追问链】→ 为什么不用 shared_ptr（先于标准、弱语义可裁剪）？→ 悬垂 wp 的 promote 语义？→ 死亡通知与 ref 归零的先后（ref 尽→node 销毁→BR_DEAD_BINDER）？

Q9.6 PendingIntent 的"代执行"语义？

【深度解答】它打包"意图+创建者身份+权限"成 IIntentSender 凭据（PendingIntentRecord.java:54 存在 AMS）；持有者（通知/闹钟/Widget）调用 send 时，**以创建者身份执行**（除非 FLAG_NO_CREATE/代发限定）。安全三问：可取消（cancel 后 send 报 Canceled）；可变性（13 默认 FLAG_IMMUTABLE，防持有者篡改 intent）；越权（转交即让渡身份——只交给可信方）。〔册4・九〕

【追问链】→ 为什么通知点击不走普通 Intent（通知由 SystemUI 进程发出，无权冒充你）？→ immutable 的例外修改面（setActivityStack? 13 只允许 bundle 更新器显式放行）？→ AlarmManager 的 PendingIntent 与 App 重启（凭据在 AMS，重启后失效需重注）？

Q9.7 onActivityResult 的结果什么时候回来？

【深度解答】目标 setResult+finish → AMS 记账 → 发起方重新可见（resume）时以 **ActivityResultItem**（与 LaunchActivityItem 同族事务）下发 → handleSendResult → onActivityResult——**结果到达时机=发起方回前台**，中间隔着整个生命周期流转。这是"事务机器"复用的又一现场：回传不是新机制，是 ClientTransaction 家族多了一个 Item。〔册4・九〕

【追问链】→ 多层启动链的结果逐级回传顺序？→ activityResultRegistry（Jetpack）解决什么（分层封装 requestCode）？→ 单结果 API（ActivityResultContract）之下还是它吗（是）？

Q9.8 动态注册的广播为什么在主线程回调？

【深度解答】LoadedApk 为每个"Context+Receiver"建 ReceiverDispatcher（LoadedApk.java:107 内部类）：IIntentReceiver 的 binder stub 收到 BroadcastQueue 分发（binder 线程）→ 立即 post 到绑定的 Handler（默认主线程）→ onReceive。**dispatcher 是 binder 世界到 Handler 世界的桥**，同族的 ServiceDispatcher 管绑定。推论：onReceive 主线程别重活（goAsync 书4 §4.2）；泄漏根源=未 unregister 导致 dispatcher 与 binder 强引用链常驻。〔册4・九〕

【追问链】→ goAsync 的 PendingResult 在哪个线程计时（超时线不变）？→ LocalBroadcast 为什么不走这套（进程内直接分发）？→ manifest 接收者有 dispatcher 吗（无——AMS 直接经 ApplicationThread 分发）？

Q9.9 视频为什么用 SurfaceView 而不是普通 View?

【深度解答】SurfaceView 拥有独立 Surface（SurfaceView.java:239 构造即申请，updateSurface :154-160 独立控制流）：视频解码线程直接上屏，帧率与 UI 滚动解耦、不占 UI 帧预算；且常走 Overlay 直通（书5 §8）省一次合成。普通 View 必须并入窗口 Surface，跟随 Choreographer 节拍——高帧率流会被 UI 帧率钳制。TextureView 折中：可动画可变换但多一次 GPU 拷贝。〔册5・九〕

【追问链】→ "打洞"（setZOrderedOnTop=false）与洞上盖 UI 的合成代价？→ BLASTBufferQueue（12+）改变了什么（提交流与事务对齐）？→ 相机预览选哪个（SurfaceView，低延迟直通）？

Q9.10 属性动画比 View 动画"真"在哪?

【深度解答】View 动画只改 draw 矩阵（视觉骗术，位置与响应区不动）；属性动画经 AnimationHandler（AnimationHandler.java:24/:83）挂 Choreographer 的 **CALLBACK_ANIMATION** 车道，每个 vsync 插值→**改真实属性**→invalidate→TRAVERSAL 车道重测布局。车道顺序（动画先于遍历，书5 §4.1）保证同帧闭环。更快的变体：RenderThread 动画（RenderNode 属性直改不重录，书5 §8）。〔册5・九〕

【追问链】→ 插值器/估值器的扩展点？→ 为什么动画中频繁 GC 会掉帧（每帧对象分配→池化 ValueAnimator）？→ Choreographer 车道与 doFrame 补偿的协作（书5 §4.1）？

第十章 渲染深潜题（10 题）

Q10.1 BufferSlot 四态各自的所有权?

【深度解答】FREE（队列所有，可分配/复用）→DEQUEUED（生产者正在写）→QUEUED（写完待消费）→ACQUIRED（消费者持有）→release 后回 FREE；SHARED 为相机持续流旁路。状态即所有权、所有权即锁边界（BufferSlot.h:53-73 注释表）。〔册8・三〕

【追问链】→ dequeue 拿到的 fence 是谁的（上一任读者的 release）？→ 三缓冲为何是四个角色的最小完备（写/排/读/垫）？→ 丢帧发生在哪个态（QUEUED 被主动收回）？

Q10.2 dequeueBuffer 的背压在哪实现?

【深度解答】waitForFreeSlotThenRelock 循环（BufferQueueProducer.cpp:411-446）：无 FREE 且未超 maxDequeuedBufferCount → 等待 release；超限则丢最老 QUEUED。背压不是协议而是**状态机内的等待**——生产过快自然被压回消费速度。〔册8・三〕

【追问链】→ maxDequeued 怎么谈判（生产者申请、SF 按内存预算点头）？→ 等分配与等 release 两种等待的区别？→ App 侧 dequeue 卡住的排查（dumpsys --dump-buffer 槽态）？

Q10.3 BLAST 解决了旧 BufferQueue 模型的什么病?

【深度解答】三病：内容与几何双通道错配（新内容旧位置闪烁）、队列语义与事务语义纠缠、帧完成回执粗粒度。BLAST 每帧组一个 Transaction（buffer+几何+回执）原子 apply（BLASTBufferQueue.cpp:635 起），帧责任链三段回执（latched/acquired/presented）成为 FrameTimeline 数据源。〔册8・四〕

【追问链】→ 影子队列的作用（提交速率与生产速率解耦+积压丢帧）？→ desiredPresentTime 谁消费（SF commit 按预约投放=帧 pacing）？→ 与旧模型并存的场景（媒体个别路径）？

Q10.4 GraphicBuffer 怎么做到一份内存三处用?

【深度解答】Gralloc allocator 出 dma-buf；App（画）/SF（合成）/控制器（扫描）经 mapper 各自 lockAsync 映射同一 bufferId——物理页一份、虚拟视图三份。usage bits 决定落堆（GPU 纹理堆/扫描输出堆），堆错了 Overlay 就免谈。〔册8・二〕

【追问链】→ stride 是什么坑（行距≠宽×像素，算错=斜纹）？→ 跨进程传的是 fd（Binder FDA 通道）还是地址？→ 懒分配何时发生（首次需要新槽时，BufferQueueCore 请求）？

Q10.5 SurfaceFlinger 一帧的两个阶段各干什么?

【深度解答】commit（commitTransactionsLocked:3119）：应用事务、latchBuffer（取每层最新 QUEUED，fence 未到用旧帧——帧率优先）、算脏区；composite（:2164）：CompositionEngine 定合成计划→HWC validate/present→收 present fence。两阶段=世界结算与像素搬运分离。〔册8・六〕

【追问链】→ latch 旧帧算丢帧吗（该层 stale 不整帧 jank）？→ validate/present 为何两阶段（先问硬件后用，回退 GPU 才来得及）？→ offscreen 层走哪（commitOffscreenLayers:3512，自节拍）？

Q10.6 Overlay 直通为什么省电？决策怎么做?

【深度解答】直通=控制器直接采样各层（无 GPU 参与、无中间 FB 写回、少一次搬运）——视频常驻场景省一个量级。决策三步：候选层清单→HWComposer validate 硬件点名（受层数/混合/缩放/格式能力限制）→CLIENT 层 RenderEngine(Skia) 合成成一层再直通。逐帧动态决策是关键。〔册8・七〕

【追问链】→ 加半透明遮罩后视频为何变 CLIENT（混合超能力）？→ Sideband 与 Overlay 的区别（绕 SF 直叠的特权通道）？→ GPU 合成的画质差异哪来（Skia 路径才做 tone-map/色彩管理）？

Q10.7 acquire/release/present 三种 fence 的方向与用途?

【深度解答】acquire：生产者→消费者"写完了"；release：消费者→生产者"读完了"；present：控制器→SF"真上屏了"。Fence 是内核同步对象的 fd 封装（Fence.cpp:48 wait/:90 merge），跨进程走 Binder fd 通道——不轮询不忙等，依赖图由内核执行。〔册8・八〕

【追问链】→ dequeue 的 fence 为何是 release（等上家读完才能写，否则撕裂）？→ merge 什么时候用（每层 acquire 合一路再合成）？→ "presented 没生效"查什么（present fence+FrameTimeline）？

Q10.8 EventThread 的"按需分发"是什么?

【深度解答】EventThread.threadMain（:434-470）不是广播塔：每条 connection 有独立 vsyncRequest（None/Single/Periodic），App 注册 Choreographer 回调后才收；没人要的节拍不产生开销。SF 与 App 各走不同 EventThread 实例（相位错开）。〔册8・六〕

【追问链】→ BitTube 是什么（connection 的 socket 对，stealReceiveChannel:179 交给 App）？→ idle 时 vsync 轨为何稀疏（无人订阅即停发）？→ 帧率投票在哪采样（composite 后 LayerHistory）？

Q10.9 13 的帧率策略怎么决定全局刷新率?

【深度解答】LayerHistory 采每层提交间隔→启发式分层（活跃/空闲）→结合 FrameRateOverrideMappings 覆盖表提案→DisplayModeController 切面板模式。三特性：投票单位是 Layer（一个视频层可拉整机 120Hz）；切换有迟滞（防抖）；触摸即时提速（Input 与 Scheduler 联动）。〔册8・七/一〕

【追问链】→ 24fps 视频为何多数机型仍高刷（LFD 未普及，仅部分屏做层帧率）？→ 多屏刷新率（per-display 独立策略）？→ 游戏锁帧的正确姿势（desiredPresentTime+Surface.setFrameRate 而非睡线程）？

Q10.10 录屏/投屏的合成节拍与主屏一样吗?

【深度解答】不一样：VirtualDisplay 是"没有面板的屏"，composite 输出到自己的 BufferQueue（请求方消费），节拍自定（录屏常 30fps 桶）；镜像模式则是主屏 composite 的旁路复制。投屏卡顿查 VirtualDisplay 消费端（协议侧 release 是否及时），别盯主屏 vsync。〔册8・六〕

【追问链】→ 镜像与独立输出的代价差（复制 vs 双 composite）？→ 安全录屏（DRM 层）为何黑块（secure 层禁捕获）？→ 副屏独立应用的合成（per-display Layer 树）？

第十一章 后台与电源题（10 题）

Q11.1 本树 JobScheduler 源码在哪？这个迁移说明了什么?

【深度解答】frameworks/base/apex/jobscheduler/（com.android.scheduling APEX 实体）——Job/Alarm/Doze/AppStandby 整族迁入 Mainline：**调度策略高频演进且无需整机 OTA**。对照 Power/Thermal 留守 system 分区（与启动/硬件强耦合）。路径变化即架构演进的化石。〔册9・一〕

【追问链】→ Mainline 化的条件（策略可独立演进/接口稳定）？→ APEX 回滚对调度的意义（两次启动失败退旧策略）？→ 老代码路径找不到了怎么办（find 按 className 全树定位）？

Q11.2 Job 为什么用 bindService 执行?

【深度解答】JobServiceContext（:79）implements ServiceConnection：bind 意味着系统握连接——约束失效（拔线/断网）即 unbind 收编，超时有硬顶；startService 出去的进程系统只能杀不能优雅停。**bind=生杀权换稳定性**。〔册9・三〕

【追问链】→ onStartJob 返回 false 的语义（无事可做即完）？→ jobFinished 与 stopSelf 差异（前者记账触发重排）？→ 崩溃后的 Job 命运（退避重试）？

Q11.3 一个 Job 从 schedule 到执行的完整链?

【深度解答】四站：scheduleAsPackage(:1104) 入账 JobStore（落盘）→各控制器订阅约束→maybeQueueReadyJobsForExecutionLocked(:2406) 从就绪池挑队（十二控制器"与"全绿）→QuotaController:887 最后一闸→JobServiceContext bind 拉起→onStartJob。排障即按站查 dumpsys。〔册9・三〕

【追问链】→ 十二控制器各管什么（速查表册9 §3.2.1）？→ 约束执行中失效会怎样（立即收回）？→ 就绪但不出队的两大原因（配额/桶）？

Q11.4 Doze 七态怎么走？维护窗为什么递减?

【深度解答】ACTIVE→INACTIVE（灭屏拔电 30min）→IDLE_PENDING（60min）→SENSING（运动检测 4min，有动即回）→[LOCATING 30s]→IDLE（断网/挂 wakelock）↔IDLE_MAINTENANCE（首个约 5min 递减开窗放同步）。递减=同步需求随静置递减的假设；QUICK_DOZE 旁路跳过定位直入深睡。源码内嵌官方 DOT 图 :161-208。〔册9・二〕

【追问链】→ 充电为什么全退（ACTIVE，充电无续航焦虑）？→ 白名单三层（全量/除 Idle/用户可见）谁豁免什么？→ Light/Deep/Quick 三档的适用差异?

Q11.5 AllowWhileIdle 的 9 分钟配额是什么设计?

【深度解答】打穿 Doze 的唤醒必须稀缺（否则 Doze 名存实亡）：每 App 每 9 分钟一次豁免窗口。这是"API 即政策"的典范——框架不禁止能力，只给能力定价，把开发者逐向宽松窗口与 Job 化。〔册9・四〕

【追问链】→ setExact 与 setWindow 的合并账（精确钟阻止近邻合并）？→ 闹钟耗电排行计的是"钟"还是"源"（合并后唤醒源）？→ Doze 中非 wakeup 钟的命运（延至维护窗 :2464）？

Q11.6 Standby 五桶各自砍掉什么？怎么升降桶?

【深度解答】ACTIVE 全额→WORKING_SET Job 约半→FREQUENT 约 1/8→RARE 几乎无→RESTRICTED 每日窗口+FGS 受限。升：用户点开一步回 ACTIVE（:726/:767）；降：15 天 PAR 窗口渐进+预测模型。三个下游：Job 频率/Alarm 窗/后台网络。〔册9・五〕

【追问链】→ `am set-standby-bucket` 测试法？→ RESTRICTED 下推送的命运（依赖高优先 FCM/厂商通道）？→ 桶与冻结的关系（先桶限制、深桶再冻书4 §九）？

Q11.7 QuotaController 的双窗口计费防什么博弈？

【深度解答】10 分钟短窗防爆发（攒额度一次跑飞）、24 小时长窗防长期占用；计"执行"不计"排队"（被卡不扣钱，鼓励快速幂等）；前台/充电/维护窗免检。超账=排队不拒绝——冲突化为时序。〔册9・三〕

【追问链】→ dumpsys 哪段看账本（Job quotas）？→ expedited 任务怎么绕（前台级配额池）？→ WorkManager 在这层的增值（重试/链式/兜底通道）？

Q11.8 灭屏后进程还在跑吗？AOD 是什么态？

【深度解答】在——ASLEEP 只关屏；CPU 挂起需无任何 SuspendBlocker（wakelock 汇总后）。AOD 是 DOZING 态：屏走低亮驱动路径、CPU 可睡。13 新 PowerGroup 让每显示组独立唤醒度（外屏灭内屏亮合法化）。〔册9・六〕

【追问链】→ PARTIAL_WAKE_LOCK 灭屏为何仍拦挂起（拦的是 CPU 不是屏）？→ "先记账后结算"模式（userActivityNoUpdate :1932 命名即架构）？→ LowPowerStandby 治什么（息屏外设极简，与 Doze 分工）？

Q11.9 热缓解三层怎么分？应用该做什么？

【深度解答】三层：HAL 直管频点（最快）、framework 联动（AOD 关/亮度帽/帧率降——书8 帧率的负输入）、应用回调（THERMAL_STATUS_*）。:267 shutdownIfNeeded 是安全兜底。应用正解：MODERATE 起自觉降级（降帧/降码率），而非硬扛到 SEVERE。〔册9・七〕

【追问链】→ `cmd thermalservice override-status` 测试法？→ 热与性能模式的借贷关系（高性能=借热，降频=还债）？→ 温度流有哪些（CPU/GPU/BATTERY/SKIN 各自 severity）？

Q11.10 面试串联：手机放一晚，系统做了什么？

【深度解答】系统级叙事：灭屏→PowerGroup 倒计时到→ASLEEP（AOD 走 DOZING）→DeviceIdle 走 INACTIVE→SENSING 确认静止→IDLE（断网挂锁，缓存 App 逐步冻结书4 §九）→维护窗定期开闸（Job/Alarm 冲配额窗口）→天亮拿起：运动/亮屏→ACTIVE，冻结逐层解冻，你的 App 一步回 ACTIVE 桶。**这一题串起本册五章+书4 冻结+书8 帧率**——是后台与电源的压轴叙事。〔册9・全〕

【追问链】→ 整晚耗电怎么归因（battery historian+wakeup_sources+alarm 源）？→ 为什么放一晚再打开 App 感觉"冷"（冻结态恢复=准冷启动）？→ 厂商"智能省电"与 AOSP 差在哪（桶策略厂化+更激进冻结）？

第十二章 网络栈题（10 题）

Q12.1 本树 Connectivity 源码在哪？模块化动机？

【深度解答】packages/modules/Connectivity/（Mainline，com.android.tethering APEX 分发）；NetworkMonitor 在独立 NetworkStack 模块；netd 在 system/netd/server/（main.cpp:111）；Vpn.java 仍在 frameworks/base（:179 活化石）。动机：运营商/安全补丁急速分发；NetworkAgent AIDL 稳定是前提。〔册10・一〕

【追问链】→ Vpn 为何留守（挂靠 IConnectivityManager 接口）？→ NetworkStack 为何独立（验证逻辑被 Wi-Fi/CS/Tethering 复用）？→ 老路径找不到类怎么办（find 按 className 全树）？

Q12.2 默认网怎么选出来的?

【深度解答】请求（NetworkRequest 能力声明）→ 供给（NetworkFactory 各网种）→ NetworkAgentInfo（活网络运行时视图：agent+FullScore+linkProperties）→ NetworkRanker 在满足能力的网里挑分最高；分数变化触发 rematch，老网 linger（默认 30s 宽限吸收毛刺）。〔册10・二〕

【追问链】→ Wi-Fi 分通常 60 蜂窝 50 的加权项（TRUSTED/NOT_METERED/验证态）？→ linger 期间 App 连接走哪（老网，TCP 不断）？→ onAvailable/onLost 成对吗（回调语义）？

Q12.3 四探针各测什么？部分连通语义?

【深度解答】NetworkMonitor（NetworkStack 模块 :225 extends StateMachine）四探针（:29-33）：PROBE_DNS（53 通否）/HTTP（204 探测+302 判门户）/HTTPS/PRIVDNS（DoT 可达+证书）。302→CAPTIVE_PORTAL（弹登录页）；HTTPS 不通 HTTP 通→PARTIAL（"无法上网"提示但 fallback 可用）。验证异步：先给网后验网，结论回来重评分。〔册10・三〕

【追问链】→ 为什么连上 Wi-Fi 标无法上网仍能用（PARTIAL 语义）？→ 探测流量走哪个网（当前网——自举）？→ 强制门户登录页谁弹（CaptivePortalLogin 模块）？

Q12.4 eBPF 流量记账全链?

【深度解答】TrafficStats.setThreadStatsTag→socket 创建时 tagSocket→内核 eBPF 写 uid_tag_map（cookie→uid+tag）→每包 cgroup hook 查 cookie 累加 per-uid-tag map→TrafficStats.getUidRxBytes 经 binder 读 map。BpfNetMaps.java:37 是 Java 门面。uid 部分内核核对（伪造 tag 逃不掉 uid 账）；iptables 只剩 UID 隔离。〔册10・四〕

【追问链】→ 为什么不用 iptables 计数（O(rules) vs map O(1)）？→ Tethering offload 同机制吗（BpfCoordinator 硬件转发）？→ Settings 流量页数据源（这套 map 快照）？

Q12.5 为什么 DNS 必须过 netd?

【深度解答】两个原因：多网络按调用方绑定网选 DNS 服务器（只有系统知道该用哪张网）；bionic 是 C 库无 binder C API，且 fork 的子进程禁 binder（书2 呼应）。链：

getaddrinfo→/dev/socket/dnsproxyd→netd resolver（per-network 缓存+A/AAAA 并发+happyeyeballs）。〔册10 · 五〕

【追问链】→ Private DNS STRICT 失败为什么断解析（用户明示要加密，安全>可用）？→ DoH 系统支持吗（默认 DoT；DoH 是 App 库行为）？→ DNS 缓存在哪（netd per-network）？

Q12.6 VPN 数据面一个包的双程?

【深度解答】去：App connect→路由命中 tun0→包入 tun 队列→VPN App read(fd) 读明文 IP 包→加密→protect 标记的 socket 发物理网→服务器。回：反序 write(fd)→内核投递 App。Vpn.establish（frameworks 侧 :1504）建 tun+路由劫持；protect 防递归（隧道流量再进 tun=自套）。〔册10 · 六〕

【追问链】→ 为什么 protect 必须显式（框架无法识别隧道流量目的）？→ always-on 与 lockdown 区别（重连 vs 无 VPN 断网）？→ per-app VPN 按 uid 路由的坑（clone profile uid 变更）？

Q12.7 多网络并发时 App 怎么选网?

【深度解答】默认走 defaultNet；App 可 bindSocketToNetwork 显式绑定（下载器双路并发）；13 的 ProfileNetworkPreference 按 profile 分流（办公走企业 APN）。切网时已连接的 TCP 不断（linger 宽限），新连接走新默认网。〔册10 · 二〕

【追问链】→ bindProcessToNetwork 与 bindSocket 区别（进程级 vs socket 级）？→ CONNECTIVITY_ACTION 里 EXTRA_NO_CONNECTIVITY 语义？→ 多开下载的最优策略（双 NetworkRequest+各自 bind）？

Q12.8 Tethering 为什么能省电?

【深度解答】BpfCoordinator 下行 offload：eBPF 规则让硬件直接转发包不上 CPU（传统 NAT 每包两次软中断+netfilter 全链）。IpServer 每下游接口一实例（DHCP/NAT/限速）。省电与限速同 map 体系。〔册10 · 四〕

【追问链】→ offload 不支持时回退什么（内核 netfilter NAT）？→ 热点限速在哪实现（tc/eBPF）？→ 客户端统计哪来（同 eBPF map）？

Q12.9 Wi-Fi 框架在哪、怎么和 CS 协作?

【深度解答】packages/modules/Wifi（独立 Mainline）：WifiService 管 STA/AP/P2P 状态机与 HAL；对 CS 供给 NetworkAgent（"我有个 Wi-Fi 网，分 60"）——**供给与决策分离**：Wi-Fi 模块不知道谁是默认网，CS 不知道 Wi-Fi 怎么连。〔册10 · 二〕

【追问链】→ Wi-Fi 分数谁定（WifiScoreReport：RSSI+链路质量）？→ 飞行模式瞬间发生了什么（CS 广播→各 Factory teardown）？→ Wi-Fi 直连为何是独立网络（p2p 接口独立 netId）？

Q12.10 面试串联：输入网址到页面显示，网络层走几步?

【深度解答】压轴串联：DNS（netd resolver：DoT 检查→缓存/并发查询）→路由（defaultNet 的接口）→netfilter/eBPF（记账+UID 策略）→TCP/TLS 握手→HTTP；期间 CS 随时可能重评分换默认网（linger 保护长连接）；VPN 场景再叠 tun 双程。**一题串起本册六章+书3 Binder（DNS 的 socket 与 binder 边界）**。〔册10 · 全〕

【追问链】→ 全程哪些步在用户态哪些在内核？→ 首字节慢的三大嫌疑（DNS/握手/服务器）怎么分层定位？→ 与书8 渲染的"首帧"串成完整体验链？

第十三章 存储与加密题（10 题）

Q13.1 FBE 与 FDE 的本质区别？

【深度解答】粒度：FDE 全盘一钥（解锁前全系统不可用）；FBE 按目录分域（DE 开机即解/CE 解锁后解）——"重启后闹钟响但微信看不到"成为可能。FsCrypt.cpp:842/858/873 是 DE 侧三锚点（寻址/建目录/绑策略）。〔册11 · 一〕

【追问链】→ 每文件密钥怎么派生（目录密钥 HKDF 派生）？→ metadata encryption 与 FBE 什么关系（文件名/目录树另一层）？→ FDE 遗留代码在哪（EncryptInplace.cpp 活化石）？

Q13.2 改 PIN 为什么不用重新加密 /data？

【深度解答】Synthetic Password 间接层：CE key 由 SP 派生，PIN 只是"解锁 SP 的钥匙"——换 PIN 只换 SP 的 KEK 包装（Gatekeeper 签名密钥+Weaver 双份）。注册新指纹=给 SP 加钥匙。LockSettingsService.java:202 域。〔册11 · 二〕

【追问链】→ SP 存哪被谁包（LockSettingsStorage 双份）？→ 生物特征参与密码学吗（只是另一把钥匙，不进密钥派生）？→ 企业策略改密的差异（ManagedProfile 独立 SP）？

Q13.3 密钥链五层各在哪？

【深度解答】①PIN（用户）→②Gatekeeper HAL（gatekeeper/1.0 TEE 校验）→③Synthetic Password（LockSettingsService）→④DE/CE Key（vold KeyStorage.cpp，FsCrypt:195 KeyAuthentication）→⑤文件密钥（内核 fscrypt HKDF）。TEE 边界：PIN 本体不出框架加密包装。〔册11 · 二〕

【追问链】→ Gatekeeper 为什么冻结 HIDL 1.0（接口极简无演进需求）？→ Weaver 防什么（离线爆破硬件计数器）？→ StrongAuth 重启必须 PIN 的本源（生物数据不可信链）？

Q13.4 keystore2 的 Rust 重写换来了什么？

【深度解答】并发安全（密钥操作竞态=安全事故）+新授权模型（条目 grant/用法 KeyGenParameterSpec/用户认证三层）+attestation 完整化。service.rs:54 按 KeyMint 实例（TEE/StrongBox）路由。App 只有句柄无私钥——银行 App"导不出密钥"的保证。〔册11 · 三〕

【追问链】→ StrongBox 与 TEE 分级（独立芯片抗物理，高价值密钥才配）？→ attestation 证书链给谁（Play Integrity 远程证明）？→ keystore1 的旧 API 还在吗（兼容层）？

Q13.5 direct boot 期间什么能用什么不能?

【深度解答】DE 域可用（闹钟/电话/通知服务——vold 无口令装钥）；CE 域密文（App 私数据/媒体）。App 适配：directBootAware 组件在 DE 期跑（最小集）；常规 App 监听 BOOT_COMPLETED+USER_UNLOCKED 双广播；CE 访问解锁前是挂起不是崩溃。〔册11・四〕

【追问链】→ 重启后闹钟响但微信通知不来的机制层解释（通知服务 DE、微信处理需 CE）？→ ce_available 属性怎么查（getprop sys.user.0.ce_available）？→ 工作资料的解锁次序（主用户先）？

Q13.6 vold 的四个卷类型?

【深度解答】private（emulated /data 本体）/public（SD/U 盘挂 media_rw）/adoptable（并入加密池变内部）/stub（用户挂载视图）。VolumeManager:173 start/:442 onUserStarted；唯一写者原则：shell 改不了 vold 管的卷。〔册11・五〕

【追问链】→ adoptable 为什么格式化（并入 fscrypt 池）？→ U 盘 exe 为什么不能跑（noexec+SELinux 挂载参数）？→ FUSE passthrough（13）优化了什么（守门命中后内核直通）？

Q13.7 解锁 bootloader 后还剩几道墙?

【深度解答】四道墙（BootROM 熔丝/AVB/dm-verity/fscrypt+KeyMint）中：AVB 橙警、verity 可禁；fscrypt 照常（密钥随用户数据）、KeyMint 照常（TEE 不看锁）——attestation 标注解锁态，Play Integrity 据此识别。四层防的对象不同（防刷入/防启动篡改/防运行时改/防提取），不是重复防御。〔册11・六〕

【追问链】→ root 后银行 App 还能用吗（KeyMint 可用但 attestation 报 root）？→ dm-verity 关掉的检测面（Play Integrity API）？→ 为什么不做"解锁即熔断 TEE"（可用性权衡）？

Q13.8 密钥换机怎么办?

【深度解答】不迁移：CE/DE 密钥本机绑定；keystore2 密钥不可导出（硬件绑定，银行需新机重绑）；SP 不参与云备份。云备份的密文用云端密钥重加密——本机链与云链互不相交（云可用性与本机安全并存的设计底座）。〔册11・二・补〕

【追问链】→ 换机克隆走的什么通道（书6 D2D+云链）？→ 恢复出厂=?（密钥销毁，数据理论不可复原）？→ 企业 wipe 的强制层（KEK 销毁即全灭）？

Q13.9 OTA 数据面 Checkpoint 防什么?

【深度解答】A/B 只保系统面；应用数据被半 OTA 弄坏同样灾难。vold Checkpoint.cpp：OTA 期间 /data 写前重定向（快照），失败回滚整个数据面——与 A/B 系统面成"双保险"。〔册11・五〕

【追问链】→ 快照的粒度（块级 COW）？→ 回滚时正在写的 App 数据？→ 与虚拟 A/B 的 COW 同源吗（同机制不同对象）？

Q13.10 面试串联：描述从按下电源到微信收到加密消息?

【深度解答】终极串联：BootROM 验链（册11・六）→init 两阶段（册2）→SystemServer 起服务（册2）→vold 解 DE（册11・一/四）→锁屏→输 PIN→Gatekeeper→SP→CE 解（册11・二）→微信进程起（册4 孵化）→网络栈 DNS/路由（册10）→推送长连接（MTK/GCM）→通知（SystemUI）。**一题横跨六册**——丛书设计的终点就是能独立完成这条叙事。〔册11・全〕

【追问链】→ 每一步的"失败分支"（验证不过/解锁失败/无网）？→ 哪些步可并行（boot 动画与 PMS 扫描）？→ 把这条链讲给面试官要几分钟（三分钟版/十分钟版取舍）？

Q13.11 FBE 的 DE/CE 密钥由谁在什么时机生成、怎么挂到目录上？

【深度解答】两层钥匙、两条时间线。系统级：fscrypt_initialize_systemwide_keys（FsCrypt.cpp:435）取回/生成 device_key 装为 system DE（管 /data 根目录树），另生成 per_boot_key 写 key_raw_ref（:461-464）——重启即失的临时域。用户级：fscrypt_init_user0（:477）先备 /data/system/ce 与 de 目录（:479-482），create_and_install_user_keys（:330）给每用户生成两把独立密钥（generateStorageKey :334-335），落盘次序有讲究：CE key 先 storeKeyAtomically（:345）、DE key 后写（源码注释明言 "Write DE key second; once this is written, all is good" :349——DE 在场即宣告用户可用），最后 install_storage_key 把两把分别挂进 s_ce_policies/s_de_policies（:356-362），内核侧以 key_raw_ref 绑定目录策略。DE 用 kEmptyAuthentication 落盘（无口令——开机即装，direct boot 的底气）；CE 的解锁在 framework 侧（SP 解包后 fscrypt_unlock_user_key :752 把明文注入内核）。〔册11・一・补〕

【追问链】→ per_boot_key 给谁用（per-boot 加密目录，重启即失，适合缓存类敏感数据）？→ 用户 0 也要在 init_user0 才生成密钥吗（是——CE 要等首解锁，DE 先行：先 DE 后 CE 的时序即 direct boot 时序）？→ evict_ce_key（:552）何时触发（fscrypt_lock_user_key :781——用户锁定时把 CE 明文踢出内核）？

Q13.12 Synthetic Password 的完整派生链是什么？为什么换 PIN 不用重加密 /data？

【深度解答】PIN 从来不是密钥本体。SP 由用户凭据经 NIST SP 800-108 计数器模式 KDF 派生（SP800Derive.java:36）；"无锁屏"用户也有合法 SP——凭据为 None 时用常量 DEFAULT_PASSWORD（SyntheticPasswordManager.java:110，:1501 的 isNone 分支）。有 LSKF 的设备加 Weaver 双保险：用 SP 派生 weaver-key/weaver-pwd/weaver-token 三组个性化材料（PERSONALISATION_WEAVER_* :141-143），读 Weaver 硬件槽（WEAVER_SLOT_NAME :106）换回 token 参与解包——硬件计数器把离线爆破变成"每次尝试烧一次额度"。CE key 由 SP 再包装；改 PIN 只换 SP 的外层包装（Gatekeeper 签名 blob+Weaver 槽重配），SP 值不变→CE key 不变→文件密钥与目录内容纹丝不动。〔册11・二・补〕

【追问链】→ 为什么选 SP800-108 而非 PBKDF2（要 context/label 分域派生多把子钥：SP、weaver 材料、escrow——一套 KDF 全覆盖）？→ 换 PIN 时 Weaver 旧槽怎么办（重配新槽+作废旧槽，槽是稀缺资源）？→ 生物特征在这条链的哪一格（指纹/人脸只是"解锁 SP 的另一把钥匙"，不参与派生）？

Q13.13 keystore2 的锁定状态机三态是什么？锁屏时密钥请求怎么死？

【深度解答】super_key.rs:1216 的 enum UserState 三态：LskfUnlocked(Arc<SuperKey>)（super key 明文在内存缓存）、LskfLocked（重启后未输 PIN——加密形态在数据库、明文不在）、Uninitialized（未设 LSKF）。建密钥入口 super_encrypt_on_key_init（:705-723）按态分岔：LskfUnlocked→用 AES-256-GCM super key 加包（:716-718）；LskfLocked→ResponseCode::LOCKED 立即拒绝（:720-721）；Uninitialized→UNINITIALIZED。super key 本身分级：LskfLocked 级"对应 superencryption key 常驻内存"（:83——重启不灭，锁屏也能用的密钥类）、ScreenLockBound（:110 锁屏绑定）、BootLevel(i32)（:112 按开机级别经 boot_level_keys.rs 派生 level-zero key）。输对 PIN→LSKF 解出主 super key 进缓存→态迁移 LskfUnlocked。〔册11・三・补〕

【追问链】→ 锁屏时"已被 super 加密的密钥"请求会怎样（解包即需 super key——LskfLocked 态报 LOCKED，除非元数据标记为 NonSensitive 类）？→ BootLevel key 为什么逐级独立（防高 boot level 的 key 在早期启动阶段被诱导使用）？→ 这与 direct boot 的 DE/CE 语义怎么对齐（keystore2 的 LskfLocked 态≈CE 未解的 keystore 世界版）？

第十四章 媒体管线题（10 题）

Q14.1 AF 和 AP 的分工一句话?

【深度解答】AF（AudioFlinger.cpp:306，audioserver 进程）是混音车间（MixerThread:4908 每周期把 Track 混音写 HAL）；AP（AudioPolicyManager）是调度室（路由/音量/焦点决策，getOutputForAttr）。策略与执行分离：变化频率不同的两层各自演进。〔册12・一〕

【追问链】→ 为什么分进程放 audioserver（崩溃隔离+实时调度统一）？→ 每输出设备一条线程？→ Offload 为什么不能混音（压缩流独占输出）？

Q14.2 createTrack 全链?

【深度解答】AudioTrack::set（libaudioclient :500）→binder createTrack（AF :1023）→AF 问 AP getOutputForAttr（这个流该去哪设备）→建 Track 挂对应 MixerThread→App write 进环形 buffer（FAST track 共享内存直投）。〔册12・一〕

【追问链】→ FAST 与 normal 的 buffer 差异（共享内存 vs binder 写）？→ 音量在哪施加（软件混音乘增益/硬件直写 amplifier）？→ 拔耳机瞬间 Track 怎么迁移（AP 重路由→换线程）？

Q14.3 焦点为什么是"约定"?

【深度解答】requestAudioFocus 只是策略侧记账+广播 LOSS/GAIN，混音物理层不拦——"被夺焦停音乐"是 App 守约。系统只在铃声/通话强制降权（duckt）。流氓 App 可"抢音"正是这个设计的代价（换取导航+音乐混音等合法场景的灵活性）。〔册12・一〕

【追问链】→ GAIN_TRANSIENT vs GAIN_EXCLUSIVE？→ 通话为什么能强制（PHONE usage 特权）？→ Android O 后的延迟获取（AUDIOFOCUS_DELAY 失败即拒）？

Q14.4 三个延迟档与选型?

【深度解答】normal（~100ms：binder 写+混音周期）→fast track（~20ms：共享内存+低周期）→MMAP no-IRQ（<10ms：硬件时钟直驱，AAudio）。音游/通话选 MMAP，提示音 normal——延迟用功耗换，不是免费。〔册12・一〕

【追问链】→ MMAP 的 buffer 谁管（硬件时间戳，AF 不经手）？→ AAudio 底层仍归谁管（AF/AP——并存非替代）？→ 蓝牙为什么延迟高（编解码+协议栈缓冲）？

Q14.5 Camera2 的核心抽象?

【深度解答】异步请求-结果对：CaptureRequest（Surface+参数）→CameraService（:145）connectHelper（:1750）接入→按 HAL in-flight 节奏下发→厂商 HAL 处理 3A/ISP→TotalCaptureResult（图像+metadata"怎么拍的"）。预览=setRepeatingRequest 无限重复；拍照=插一个高分辨率请求。〔册12・二〕

【追问链】→ in-flight 上限什么定（吞吐与内存平衡）？→ metadata 为什么必要（后处理/HDR+需要曝光参数）？→ 失败的 onCaptureFailed 带什么?

Q14.6 Camera1/2/X 关系?

【深度解答】Camera1 黑箱（takePicture 一锤子）；Camera2 把 ISP 开成可编程请求队列（逐帧改参/ZSL 零快门/多摄同步/HDR+/RAW）；CameraX 是 Camera2 的易用层（生命周期感知+厂商扩展），不是新管线。〔册12・二〕

【追问链】→ ZSL 为什么只有流水线架构能做（环形缓冲回溯取帧）？→ 多摄同步谁保证（HAL sensor sync+时间戳对齐）？→ 逻辑相机与物理头（getPhysicalCameraIds）？

Q14.7 相机权限与绿点?

【深度解答】CAMERA 运行时权限+必须前台（后台/无前台服务即拒）；指示器由 SystemUI 监听 camera use 事件强制点亮——不可关闭（隐私优先于体验）。dumpsys media.camera 看客户端归属。〔册12・二〕

【追问链】→ SystemUI 怎么知道谁在用（use 事件广播 uid）？→ 双开 App 的相机归属？→ 无指示器的老设备（厂商适配差异）？

Q14.8 MediaCodec 双队列模型?

【深度解答】App↔inputBuffer 队列↔编解码器↔outputBuffer 队列↔App：池化复用（图形内存分配昂贵，从热路径挪走）；解码直投 Surface（buffer 不过 App 手省一跳）。底层 OpenMAX→Codec2（组件链，参数运行时可调）迁移中。〔册12・三〕

【追问链】→ 异步模式（7+ 回调）与同步的差异？→ 直投时 App 怎么拿到帧信息（onOutputFormatChanged+ImageReader 可选旁路）？→ HW codec 独占是什么问题（多路并发超设备上限）？

Q14.9 DRM L1/L3 与 KeyMint 关系?

【深度解答】MediaDrm 会话密钥走 KeyMint 同源硬件路径；L1=Secure Decoder 全链加密（明文不落普通内存，4K）；L3=软件解密（受限分辨率）。与 fscrypt 同哲学：密文到最后一刻。〔册12・三/册11・三〕

【追问链】→ 为什么 L1 能防录屏（安全层禁捕获，书8 §六镜像）？ → 设备怎么知道自己的级别（DRM 属性查询）？ → Widevine 与 PlayReady 并存（多 DRM 实例）？

Q14.10 面试串联：视频通话的媒体全链?

【深度解答】串联：麦克风（RecordThread）→AEC/NS（HAL/软件）→编码（MediaCodec/AV1 或 VP8）→RTP/网络（册10 默认网+可能的 VPN tun）→对端解码→渲染（SurfaceView 独立 Surface，册5 §补）→扬声器（MixerThread）；全程三个实时域以时间戳对齐。**一题串书5/8/10/12**。〔册12・全〕

【追问链】→ 三个实时域的时钟怎么对齐（音频主时钟+视频追帧）？ → 回声消除在哪层（HAL/系统）？ → 弱网降级路径（码率自适应→帧率→分辨率）？

第十五章 SystemUI 与通知题（10 题）

Q15.1 SystemUI 是系统服务吗？为什么这样设计?

【深度解答】不是——SystemUIApplication:62 extends Application，是特权 App（uid SYSTEM、platform 签名），进程 com.android.systemui。四条理由：崩溃隔离（UI 死了系统活着自动重启）、渲染独立（不拖服务线程）、可独立 OTA、内存配额独立。〔册13・一〕

【追问链】→ SystemUI 崩了会怎样（init critical 自动重启）？ → CoreStartable 模型解决什么（组件独立启停+dumpsys 分查）？ → 为什么逐步 Dagger 化（Dependency.get() 全局单例测试难）?

Q15.2 通知从 notify() 到状态栏的完整链?

【深度解答】四段：①App 建通道+Builder→notify() ②NMS:349 enqueueNotificationInternal(:6397)：通道检查→速率→排序（Extractor 链）→buzzBeepBlink ③notifyPostedListeners→binder 回调 SystemUI ④NotificationListener 渲染面板/heads-up。〔册13・二〕

【追问链】→ 排序谁决定（NotificationSignalExtractor 链：Importance/Channel/ZenMode/…）？ → 勿扰在哪拦截（ZenModeExtractor）？ → Bubbles 走同一条管线吗（BubbleExtractor 分支）？

Q15.3 "通知不显示"的排查树?

【深度解答】五步：通道开关（8+ createChannel 是前提）→勿扰模式→importance 被降→速率超限→Standby RESTRICTED 桶。`dumpsys notification` 一次看全。〔册13・二〕

【追问链】→ 厂商差异在哪层（策略默认值/厂商 Extractor 插件）？→ FGS 通知为什么总能显示（前台服务豁免）？→ 通知点击走什么（PendingIntent 书4 §补全三）？

Q15.4 最近任务的截图谁截的？什么时候？

【深度解答】TaskSnapshotController（wm/）在 Activity 切后台时经 SF screenshot 接口截图（GraphicBuffer 存储）；RecentTasks(:109) 管理任务栈。Launcher3 的 RecentsView 消费截图显示——"秒开"是先显截图再让 App 追帧的假象。〔册13・三〕

【追问链】→ 快照为什么不是实时窗口（性能+隐私）？→ 分屏两个快照？→ 13 的 snapshot 缓存策略（LRU+按键回收）？

Q15.5 Launcher3 的架构特点？

【深度解答】StatefulActivity<LauncherState>（:238）状态驱动 UI——NORMAL/OVERVIEW/WIDGET_PICKER 各声明可交互元素；LauncherModel(:87) 监听 App 装卸（LauncherApps.Callback↔书6 PMS）维护图标+widget 布局库；IconCache 跨启动持久化。〔册13・四〕

【追问链】→ 状态切换怎么实现的（每个 LauncherState 定义哪些属性）？→ 图标缓存什么时候失效（版本签名变化）？→ 预测排序谁提供（UsageStats 联动书9）？

Q15.6 CoreStartable 微服务模型怎么装配与启动？

【深度解答】SystemUI 把"大而全"拆成 N 个可插拔模块：抽象类 CoreStartable 只约定 start()（CoreStartable.java:34-42），外加 dump/onBootCompleted/onConfigurationChanged 三个生命周期钩子。装配靠 Dagger 多绑定收口：SystemUIApplication.startServicesIfNeeded（SystemUIApplication.java:191）从 SystemUIFactory.getStartableComponents/getStartableComponentsPerUser 取集合，塞进 TreeMap 按类名排序（:195-198）——启动顺序确定性（日志可对齐、bug 可复现），vendor 组件 getVendorComponent（:192-193）追加在末位。触发点在 SystemUIService.onCreate（SystemUIService.java:70）。per-user 一组独立入口 startSecondaryUserServicesIfNeeded（:216）——多用户各起一份。〔册13・一・补〕

【追问链】→ 不排序会怎样（Dagger set 顺序未定义，通知/状态栏/分屏的初始化次序在构建间漂移）？→ 某个 CoreStartable.start() 抛异常会怎样（启动期全串行，一个 fail 拖累后续——所以 start 内部普遍 try-catch+降级）？→ 新增一个系统 UI 模块的最小改动（写 CoreStartable 子类 +Dagger into-set 绑定，无需改任何清单文件）？

Q15.7 HUN（横幅）判定链有哪些闸门？为什么"响"和"弹"是两套逻辑？

【深度解答】分岔点 shouldHeadsUp（NotificationInterruptStateProviderImpl.java:166）：dozing 走 shouldHeadsUpWhenDozing（:252，pulse_on_notification+AOD 省电），awake 走 shouldHeadsUpWhenAwake（:185）的闸门链：mUseHeadsUp 总开关（:188）→canAlertCommon（:193，实现 :289：组抑制 suppressAlertingDueToGrouping :298、刚弹过 fullscreen intent :310、锁屏隐藏策略 :315）→canAlertAwakeCommon（:196，suppressors :332）→isSnoozedPackage（:201，同包轰炸被 snooze）→bubble 已在 SHADE 不重复弹（:206-208）

→DND shouldSuppressPeek（:212）→重要性门槛 IMPORTANCE_HIGH（:217）→屏幕在用判定（:220-233）→mSuppressors 逐个表决（:235）。〔册13・二・补〕

【追问链】→ NMS 的 buzzBeepBlink 与 SystemUI 的 HUN 判定为什么分两层（服务端决定"要不要发声震动"，UI 端决定"要不要霸屏"——权限与策略分治）？→ fullScreenIntent 什么时候真拉 Activity（shouldLaunchFullScreenIntentWhenAdded :179：非 HUN 或锁屏态——闹钟/来电的特权通道）？→ 排查"为什么不弹横幅"的最快路径（mLogger 每道闸一条日志，dump 里直接给出 NoHeadsUp 原因枚举）？

Q15.8 NotifCollection 的"账本"模型长什么样?

【深度解答】单账本+事件重放。NotifCollection（NotifCollection.java:140）持有 mNotificationSet（Map<key, NotificationEntry> :149），上游 attach 一个 GroupCoalescer（:184）把 group 批量到达的事件合并。每笔变动都是"记队列→统一重放"：onNotificationPosted（:393）=postNotification（新条目产生 InitEntryEvent/BindEntryEvent/EntryAddedEvent :452-455，已存在则 BindEntryEvent 更新）+applyRanking+dispatchEventsAndRebuildList（:589）；onNotificationRemoved（:412）不直接抹账——先记 mCancellationReason（:423）再 tryRemoveNotification（:428），让下游 renderers 看到完整生命周期（动画/收尾有据可查）；账上无此 key 的 removal 只记日志丢弃（:419，TODO b/160008901 未来要抛异常）。〔册13・二・补〕

【追问链】→ 为什么"事件队列+重放"而不是直接改状态（主线程单写者、单测可断言事件流、渲染层所见即完整历史）？→ cancellationReason 都有什么（用户划掉/app 撤回/超时/importance 降 NONE……renderers 据此选动画与是否 buzz）？→ GroupCoalescer 合并解决什么（同 group 秒级连发 N 条只触发一次重建，防排序抖动）？

Q15.9 NMS 入队要过哪三道闸?

【深度解答】enqueueNotificationInternal（NotificationManagerService.java:6397/:6404）。闸一身份：resolveNotificationUid（:6423）把 opPkg/callingUid 解析成通知归属 UID，解析不出（不被信任的代发）立即 SecurityException（:6424-6426）。闸二合规：checkRestrictedCategories（:6430）拒受限 category、fixNotification（:6434）补桩修形（FGS 修正失败抛 SecurityException :6437-6439）、FGS 通知先过 mAmi.applyForegroundServiceNotification（:6448，UPDATE_ONLY 且未在显→直接吞）。闸三通道：getConversationNotificationChannel（:6472-6474）查不到即 toast 开发者警告+静默 return（:6475-6496）——8.0 之后无通道=物理静音。过关后还有配额闸 checkDisqualifyingFeatures（:6808）：单包入队速率超限（getAppEnqueueRate :6836）与单包存量 ≥ MAX_PACKAGE_NOTIFICATIONS=50（:361，:6853）都直接丢弃。〔册13・二・补〕

【追问链】→ 为什么 FGS 重要性补偿（保底 IMPORTANCE_LOW）放在通道闸之后（:6508 起——importance 先从通道读出才能补，用户锁死 importance 的除外）？→ 被速率/数量闸拒绝时调用方拿到什么（notify 正常返回、静默丢弃——只能 logcat/dumpsys notification 的 registerOverRateQuota/OverCountQuota 识别）？→ 三道闸哪道最常误伤（通道闸——应用升级重

装后通道被删或未建）？

第十六章 HAL 实战题（10 题）

Q16.1 新 HAL 用什么 IDL？为什么？

【深度解答】AIDL。工具链归一（复用 /dev/binder+servicemanager+一套编译器/调试）；隔离由 vendor 分区+VINTF+SELinux 保证（非语言）。HIDL 冻结维护存量（camera 等）。〔册14・一〕

【追问链】→ "隔离"来自哪（分区+策略，非 IDL）？→ HIDL 走哪个设备（/dev/hwbinder+hwservicemanager）？→ 读码怎么区分（目录 aidl/ vs 1.x/）？

Q16.2 AIDL HAL 五步是什么？

【深度解答】①定义 .aidl 接口（aidl_interface 自动四后端）②参考实现（Power.cpp 模板）③Android.bp（cc_binary+init_rc+vintf_fragments+vendor:true）④rc+VINTF XML+SELinux 三件套⑤客户端 ServiceManager.getService→Stub.asInterface→调用。〔册14・二〕

【追问链】→ Android.bp 四个关键属性各干什么（init_rc/vintf_fragments/vendor/relative_install_path）？→ SELinux 三件是什么（.te+file_contexts+service_contexts）？→ VINTF 匹配失败会怎样（构建期拒绝或启动失败）？

Q16.3 AIDL 接口版本为什么冻结在 aidl_api/ 目录？

【深度解答】OTA 兼容性保证：老 framework 必须能调新 HAL——stable AIDL 以版本目录（aidl_api/1,2,…）冻结接口签名，新增方法必须开新版本目录。framework 与 vendor 的 compatibility_matrices 在构建期对齐（不匹配=拒绝构建）。〔册14・二/五〕

【追问链】→ 新增一个方法怎么升版（新开 aidl_api/N 目录）？→ 删方法呢（不允许——只能 deprecate）？→ VINTF 的 manifest.xml vs matrix.xml 各是谁的（vendor 声明/framework 要求）？

Q16.4 HAL 的 init class 为什么不是 main？

【深度解答】class hal（或 early_hal 更早）：HAL 必须在 framework 服务之前就绪（书2 §四 late-init 的 hal 分支先于 main）。framework 服务启动时（如 AMS）需要立刻可用 HAL（如 PowerManagerService 要调 Power HAL）。〔册14・二〕

【追问链】→ early_hal 与 hal 差别（启动时机更早，如 keymaster 解密前置）？→ rc 里 user/group 设错什么后果（HAL 无权限读写 sysfs）？→ onrestart 会重启哪些相关服务？

Q16.5 从 PowerManager 到内核的完整跳？

【深度解答】PowerManager.setPowerSaveMode→IPowerManager.Proxy→BinderProxy→驱动（书3 十跳）→vendor 进程 binder 线程→BnPower::onTransact→Power.cpp setMode→写 sysfs→内核 cpufreq 生效。〔册14・三〕

【追问链】→ HAL 的 binder 线程模型（与 framework 同一套 ProcessState/IPCThreadState）？→ sysfs 写失败谁兜底（HAL 返回错误→framework 降级）？→ Power HAL 的 hint session（ADPF 自适应调频）？

Q16.6 vendor_dlkm 与 boot 的关系?

【深度解答】GKI（通用内核镜像）：boot 分区只装 Google 签名内核；厂商差异（.ko 内核模块）走 vendor_dlkm 分区、设备树走 dtbo——内核碎片化治理的"镜像版 Treble"。〔册14・四〕

【追问链】→ GKI 符号清单（KMI）怎么维护（abi_gki_*.xml 白名单）？→ 模块谁加载（内核 module loader + init rc 的 insmod）？→ GKI 与 Project Treble 关系（同一思想：标准件+差异化分治）？

Q16.7 passthrough 与 Sideband 各是什么?

【深度解答】passthrough（HIDL 遗留选项）：同进程直调 .so——绕过进程隔离（几乎不推荐）。Sideband（相机特权）：绕 SF 直接叠硬件（低延迟录制的底牌，书8 § 七）。主线路径永远是 binder 化 HAL。〔册14・一/五〕

【追问链】→ passthrough 为什么不推荐（崩溃连坐）？→ Sideband 的安全代价（绕过 SF 合成管控）？→ 什么场景必须 Sideband（实时取景/录像延迟<16ms）？

Q16.8 hwservicemanager 注册表的三级结构?

【深度解答】ServiceManager.h 三级嵌套。第一级 mServiceMap（:156-159）：fqName（"package::type@ver::IInterface"）→PackageInterfaceMap——源码注释直接给样例 mServiceMap["android.hidl.manager@1.0::IServiceManager"]["manager"]（:152-154）；第二级 PackageInterfaceMap（:115-145）：InstanceMap（:110，instance 名→unique_ptr<HidlService>）+包级监听器 mPackageListeners（:143）；第三级 HidlService：持 sp<IBase> 与 getService()（HidlService.h:56-59），死亡通知、pid、客户端账目挂这层。全表单线程访问无需锁（:150-151 注释）。写入路径 add（ServiceManager.cpp:311）→addImpl（:339）：先取 interfaceChain，对链上每级逐层 ACL canAdd（:350-355），重复注册做新旧 pid 对比告警（:360-375）。〔册14・五・补〕

【追问链】→ 为什么存整条继承链而不是只存叶子（castFrom 向上转型直接查表；子类注册会注销被覆盖的父类同名实例 :377 起）？→ 服务可用性回调有几档（包级 addPackageListener 与实例级 registerForNotifications 两档，先到先通知）？→ 这层 ACL 与 SELinux 的分工（SELinux 管 binder 通信权，ACL 管"谁有资格注册/查找这个名字"——名录自治）？

Q16.9 passthrough 与 binderized 在代码里的分野?

【深度解答】binderized：HAL 编成独立可执行+rc，走 /dev/hwbinder 跨进程。passthrough：调用方进程内 dlopen "X@ver-impl.so"（文件名正则 gLibraryFileNamePattern，ServiceManagement.cpp:262），dlsym 找 "HIDL_FETCH_"+接口名（:379）直取对象；openLibs（:338）先试 dlopen(nullptr) 自身（:372）再扫 hw/ 目录（:389）。两个易漏事实：①passthrough 客户端也要向 hwservicemanager 记账——registerPassthroughClient（:288），否

则 lshal 无法回答"哪个进程在用哪个 so"；②选择权在 manifest 的 transport 字段（"hwbinder"/"passthrough"），passthrough 侧有独立服务管理器 getPassthroughServiceManager（:554-577）。〔册14・五・补〕

【追问链】→ 同一个 impl .so 两种模式可共存吗（可，但同 instance 名重复注册触发 pid 对比告警——ServiceManager.cpp:360-375）？→ passthrough 的安全代价（同进程=崩溃连坐+调用方权限即 HAL 权限，最小权限原则被架空）？→ 为什么个别老 HAL 仍留 passthrough（历史接口冻结+低延迟直调；新接口一律 binderized）？

Q16.10 VINTF 兼容性判定的算法骨架?

【深度解答】VintfObject::checkCompatibility（VintfObject.cpp:587）四组交叉验证：device manifest × framework matrix（:616）、framework manifest × device matrix（:623）、runtime info（内核版本/sepolicy/avb/vndk）× framework matrix（:632）。单侧核心在 HalManifest::checkCompatibility（HalManifest.cpp:382）与 matchInstance：矩阵 required 项必须在 manifest 找到格式/包/版本/接口/实例全匹配；反向做"孤儿审计"——forEachInstance 逐个 matchInstance（:291-295），HIDL 继承版本走 childrenMap 回查（:299-311），全不中的进 unused 集合（:312）。方向性规则：required 缺失=fail；optional 无实现=pass；manifest 多余实例按矩阵 allowed/override 规则裁决。〔册14・五・补〕

【追问链】→ FCM 版本与 manifest target-level 怎么交互（设备声明 target-level 选 FCM 线，matrix 按线取要求——低 target 兼容老厂商）？→ 这套检查跑在哪几个时刻（构建期 assembly 合并校验、开机运行时查询、VTS 强制重查）？→ "矩阵 required 但设备没实现"的表象（HAL 查不到→framework 报错或 VTS fail；矩阵是声明不是加载器，不会 runtime 崩溃）？

第十七章 构建系统题（10 题）

Q17.1 Soong 的输入输出各是什么？为什么取代 Make?

【深度解答】输入 Android.bp（声明式）→输出 build.ninja。三个原因：①全树数万模块下 Kati 解析分钟级→Soong Go 并行+增量秒级②声明式无分支→变体由框架展开③androidmk 工具可自动转换（两者共存过渡）。〔册15・一〕

【追问链】→ Kati 为什么还在（百万行 Make 不可能一夜迁完）？→ Bazel 的定位（remote execution 优，试验中三引擎共存）？→ 变体轴有哪些（arch×os×link×APEX）？

Q17.2 一个 cc_binary 为什么可能展开 90+ 目标?

【深度解答】变体组合爆炸：arch(arm64/arm/x86_64/riscv64) × os(android/vendor/recovery) × link(shared/static) × APEX(95 个) ——框架自动展开所有合法组合。你声明的是"家族"，构建系统生成的是"全家福"。〔册15・一〕

【追问链】→ 怎么控制只要某个变体（target.host/target.vendor 条件）？→ 增量为什么能跳过不需要的变体（设备 variant 只编目标设备）？→ out/ 为什么那么大（90+ 目标×数万模块的中间

产物）？

Q17.3 增量构建的原理?

【深度解答】out/soong/.intermediates/ 按模块名+输入哈希缓存。`m services` 只重编哈希变化的模块及下游依赖链——编辑 ActivityManagerService.java→services.jar→system.img→adb sync，最快路径 <2min。〔册15・一/四〕

【追问链】→ m snod 干什么（只重打 system.img 不重编模块）？→ adb root+sync 的限制（userdebug 以上）？→ m module-info 查什么（模块依赖图 dump）？

Q17.4 device 树的三层继承?

【深度解答】chipset（SoC 共通，如 device/google/gs101-common）→ device（板级差异，如 redfin）→ product（机型品牌，如 redfin-userdebug）。每层 device.mk 叠加 PRODUCT_PACKAGES / PRODUCT_COPY_FILES 等变量。lunch 的目标名来自最底层的 AndroidProducts.mk。〔册15・二〕

【追问链】→ BoardConfig.mk 与 device.mk 职责差（板级硬件事实 vs 软件包配置）？→ 新建产品最小需要几个文件（AndroidProducts+BoardConfig+device.mk）？→ overlay（RRO）怎么挂（device.mk 里 DEVICE_PACKAGE_OVERLAYS）？

Q17.5 eng/userdebug/user 三个变体的差异?

【深度解答】eng：root/SELinux permissive/全调试/性能差——开发。userdebug：adb root/SELinux enforcing——调试发布。user：无 root/全限制——量产唯一合法。差异不止 root：ro.debuggable 影响 logcat/属性写。〔册15・二〕

【追问链】→ user 能开 root 吗（不能——SELinux+签名双重封）？→ userdebug 的 ro.debuggable=1 影响什么（日志更详细+非 SDK 接口警告）？→ CTS 认证必须什么变体（user）？

Q17.6 APEX 模块的 Android.bp 用什么类型?

【深度解答】apex{}：定义一个 Mainline 可升级模块（com.android.art 等）。构建产物 .apex 文件（zip 容器：apex_payload.img+AndroidManifest+签名），由 apexd 运行时 loop 挂载到 /apex/<name>@<ver>。〔册15・三/书1 §十六.2〕

【追问链】→ APEX 的回滚（两次启动失败自动退旧版，书1 §十六.2）？→ APEX 内的依赖怎么处理（classpath 隔离）？→ 哪些模块适合 APEX（策略可独立演进+接口稳定）？

Q17.7 out/ 怎么清理最安全?

【深度解答】m clean（删构建产物保留缓存骨架）；rm -rf out（彻底重建，首次 2-4h）。日常增量不需要清——空间紧张时先删 out/soong/.intermediates/ 的冷模块。〔册15・四〕

【追问链】→ "改了没生效"怎么排查（查中间产物时间戳）？→ m nothing 验证什么（bp 语法，不产出）？→ 全编最耗时的是哪个阶段（dex2oat/Jackson 生成/镜像打包）？

Q17.8 改一行 ActivityManagerService.java 最快上机的路径?

【深度解答】m services（30s-2min）→adb root→adb sync（推送 services.jar 到 /system/framework）→ kill system_server（init 自动重启加载新 jar）。注意：仅 userdebug 以上可用；system_server 重启=全部服务重启（书2 runtime restart 场景）。〔册15・一/四〕

【追问链】→ 为什么 m services 比 m 快（增量只重编受影响链）？→ framework.jar 与 services.jar 的关系（API vs 实现，书1 §五）？→ 热替换可能吗（不行——classloader 已加载）？

Q17.9 BoardConfig.mk 管什么？Android.bp 管什么?

【深度解答】BoardConfig.mk：**硬件事实**（分区尺寸/签名密钥路径/内核预编译位置/ABI 列表）。Android.bp：**构建逻辑**（模块类型/依赖/变体声明）。一个说"这块板长什么样"，一个说"怎么把软件装上去"。〔册15・二/五〕

【追问链】→ 为什么不用 bp 替代 BoardConfig（板级配置稳定且简单——mk 变量够用）？→ 签名密钥在 BoardConfig 里指定（DEFAULT_SYSTEM_DEV_CERTIFICATE）？→ 分区尺寸算错了什么后果（烧不进 flash/镜像溢出）？

Q17.10 构建系统面试终极串联：从 fork AOSP 到刷机第一行?

【深度解答】fork（repo init/sync，prebuilts 占 2/3 空间）→ lunch 选产品（device 树决定）→ m 全编（首次 2-4h：bp→ninja→clang/dex→image）→ 刷机（fastboot flashall 或模拟器）→ 开机（书2 全链）→ 恭喜你拥有了"自己读得懂的 Android"。**这条路径串起本册+书2+书14（HAL 也在 m 里出来）**。〔册15・全〕

【追问链】→ 首次全编最慢的三个模块（art/ 编译器、framework/ 大 jar、libchrome 巨型 third_party）？→ 怎么加速首次（ccache/dist 编译）？→ AOSP 每月安全补丁怎么合（repo sync+冲突解决）？

Q17.11 soong_ui 的流水线分几段、各段产物?

【深度解答】build/soong/ui/build/build.go:188 Build()：①环境体检（BecomeSingletonOrFail :199 出目录互斥锁、checkProblematicFiles :210 根目录禁 Android.mk、checkRAM :213、SetupOutDir :216、SetupPath :223）②runMakeProductConfig（:291——mk 产品变量进 soong.config；:293 注释明言 "Everything below here depends on product config"）③runSoong（:304——Android.bp→out/soong/build.ninja）④runKati（:307-309——runKatiBuild+runKatiPackage 双清单+落 KatiSuffix）⑤createCombinedBuildNinjaFile（:312，模板 :61-71 以 subninja 汇编）→ninja 执行。各段可位控开关（RunProductConfig/RunSoong/RunKati/RunKatiNinja/RunNinja :97-109，Skip* 逐段关闭；Bazel 混跑切 RunAllWithBazel :109-110）。〔册15・一・补〕

【追问链】→ 为什么 ProductConfig 必须最先（Soong/Kati 都要读它的产品变量，顺序错了下游全废）？→ help/clean 类短路参数在哪截胡（Build() 开头 :196/:206 直接 return，不走流水线）？→ installclean/dataclean 为什么也在这层（清理粒度不同：镜像产物 vs 数据面产物，:297/:303）？

Q17.12 三份 ninja 清单怎么分工？combined.ninja 干什么?

【深度解答】combined.ninja（模板 build.go:61-71）只做汇编不做规则：subninja kati 的 build 清单（:68）、package 清单（:69）、soong 的 build.ninja（:71）。三份分工：out/soong/build.ninja（SoongNinjaFile，config.go:1275-1276）——.bp 世界的模块编译/链接规则与中间产物；out/build-<product>.ninja（KatiBuildNinjaFile :1267-1268）——.mk 世界的安装/镜像规则（PRODUCT_COPY_FILES、镜像打包、installed 清单）；out/build-<product>-package.ninja（:1271-1272）——打包补充清单。收益：改 .bp 只再生 soong.ninja、改 device.mk 只再生 kati 清单，两世界增量互不拖累；combined 仅是粘合层，kati suffix 变化只改指针不重生成规则。〔册15・一・补〕

【追问链】→ 为什么不把 kati 输出并进 soong.ninja（所有权与再生条件不同，混写=任一侧变化全量重建）？→ soong.ninja 的再生触发条件（bp 文件集+目录指纹哈希比对，未变直接复用——秒进 ninja）？→ SkipKatiNinja 什么时候用（纯 Soong 变更跳过 kati，复用旧 build-<product>.ninja）？

Q17.13 APEX 从 .bp 到签名产物走几步？

【深度解答】声明层（apex.go）：apex_manifest.json（:86）、payload_type image/zip（:162）、payload 是否带 dm-verity hashtree（:177）、容器签名密钥 key（:335）；payload 依赖收集靠一张依赖标签表（:616-632：javaLib/jniLib/executable/rro/bpf/compatConfig……哪些进 payload）。构建层（builder.go）：①apexer 组装 unsigned .apex（:546/:561，命令依赖 apexer/avbtool/e2fsdroid/merge_zips/soong_zip/zipalign/aapt2 :111-126；avbtool 先给 payload 打 hash tree :54）②"Step 4: Sign the APEX using signapk"（:774-798——signapk 给 zip 容器签名）③可选压 .capex 再签（:809-825）。运行期 apexd 验容器签名→payload verity→loop 挂 /apex/<name>@<ver>；apex_pubkey 随容器分发供校验（apex.go:2265 注释）。〔册15・三・补〕

【追问链】→ 两层签名各防什么（容器 signapk 防替换+定归属；payload avb 防字节篡改+运行时 dm-verity 度量）？→ test_only_unsigned_payload 什么时候用（:181-183 测试态免 payload 签名——量产绝不允许）？→ APEX 里带 HAL 的 VINTF 责任（模块可升级但 manifest 仍要声明——Mainline 化不豁免兼容矩阵）？

第十八章 车机 AAOS 题（10 题）

Q18.1 AAOS 与 Android Auto 是什么关系？

【深度解答】完全不同的产品：AAOS 是**操作系统**（整机车机系统，packages/services/Car 全家）；Auto 是**投射 App**（手机投到车屏，手机是计算主体）。AAOS 替代整车主机，Auto 只是手机扩展。〔册16・一〕

【追问链】→ CarService 是什么进程（com.android.car APK，ProxiedService:40）？→ AAOS 能跑手机 App 吗（兼容但 UI 需适配车屏）？→ 车机 CTS 有额外用例吗（CTS-Auto）？

Q18.2 VHAL 的属性模型与手机 HAL 差异？

【深度解答】VHAL（VehicleHal:71）用**属性订阅/分发**（车速连续推送）而非方法调用——车信号本质是状态流，订阅模型天然匹配。手机 HAL（电源/传感器）多为请求-响应。〔册16・二〕

【追问链】→ 车速信号到禁止视频的链（VHAL→DrivingState→Policy→UI）？ → VHAL 的 AIDL 接口谁实现（vendor 进程，书14 模板）？ → 空调/门锁也走 VHAL 吗（是——车信号统一语言）？

Q18.3 车机电源与手机的本质差异?

【深度解答】三点：①"熄火"≠关机（深度睡眠可被 CAN 唤醒——开门/点火）②唤醒源是 CAN 而非闹钟（硬件级）③点火必须恢复到断前状态（车规）。CarPowerManagementService(:103) 的状态机（WAIT_FOR_VHAL→ON→SHUTDOWN_PREPARE→…）独立于手机 PowerManagerService。〔册16・三〕

【追问链】→ 为什么车机不能"真关机"（开门必须秒亮屏，安全法规）？ → 息屏与熄火的差异（屏关 vs 深度睡眠）？ → OTA 在什么状态跑（POST_SHUTDOWN_ENTER 前的维护窗）？

Q18.4 车机多屏怎么管?

【深度解答】CarActivityService 管多 DisplayContent（书5 §六）：仪表/中控/乘客屏各独立 ActivityDisplay。乘客可看视频而仪表禁止（安全策略：DrivingState 限制+per-display 策略）。〔册16・四〕

【追问链】→ 乘客屏焦点独立吗（per-display focus，书5）？ → 驾驶时后排可以看吗（可以——只限制驾驶员视线范围屏）？ → 仪表屏的 Activity 有特殊权限吗（系统签名+专用 displayId）？

Q18.5 CarService 为什么是独立进程?

【深度解答】三理由：崩溃隔离（车机服务死不拖系统核心）+ OEM 定制（CarFeatureController 按厂商配置启停子服务）+ 独立 OTA（车机服务升级不拖整机）。与书13 SystemUI 的设计哲学一致。〔册16・一/五〕

【追问链】→ CarService 崩了车还能开吗（能——车机≠ECU，Android 只是中控）？ → ProxiedService 懒加载什么意思（按需启动子服务）？ → 手机能装 CarService 吗（装了=变成"车机模拟器"）？

Q18.6 CarService 的"微服务容器"怎么组织子服务?

【深度解答】宿主是普通 APK：CarServiceImpl.onCreate（CarServiceImpl.java:52）→new ICarImpl（:68）→mICarImpl.init()（:73）→addService("car_service")（:77）挂上 binder 名录。ICarImpl（ICarImpl.java:101 extends ICar.Stub）构造期由 CarFeatureController（:236-238）按 OEM 配置裁剪子服务集合——不是硬编码全家桶；init()（:471）两步走：mHal.init()（:478）先立 VHAL 通道，再逐个 service.init()（:484）——与 VehicleHal.init 内部"takeProperties+init"（VehicleHal.java:256-257）同构的两级初始化。ProxiedService 懒加载让未用到的子服务不实例化。〔册16・一・补〕

【追问链】→ 为什么 init 顺序必须 VHAL 先、子服务后（子服务 init 普遍订阅车辆属性——通道未立订阅即空转）？→ CarFeatureController 关掉的特性被调用时什么表现（unsupported 返回/空实现——OEM 裁剪不等于缺位崩溃）？→ 与 SystemUI CoreStartable 的同构点（声明式清单+确定性启动序+逐服务 dump）？

Q18.7 VHAL 的属性模型：propId/areaId/changeMode 三元怎么建模一辆车？

【深度解答】VehiclePropConfig.areaConfigs 是 VehicleAreaConfig[]（VehiclePropConfig.aidl:42），每个元素一个 areaId（VehicleAreaConfig.aidl:25）+该区独立的 min/max/step——同一 propId 在多个物理区各有取值域；VehiclePropValue 每笔携带 areaId（VehiclePropValue.aidl:37）——"值"永远是"某区的值"，没有全区广播语义（要全区就发多笔）。changeMode（STATIC/ON_CHANGE/CONTINUOUS）决定订阅语义：STATIC 只能 get、ON_CHANGE 事件驱动、CONTINUOUS 按采样率推流（SubscribeOptions.sampleRate）。分区族 VehicleAreaSeat/Window/Door/Wheel 定义 areaId 的位掩码编码空间。〔册16・二・补〕

【追问链】→ 左右后视镜加热是几个属性（一个 propId+两个 WINDOW 类 areaId——车厂自定义区）？→ 主驾/副驾座椅温度怎么区分（SEAT 位掩码组合出 areaId）？→ 为什么"值必须绑区"而不是每区一个属性（属性 ID 爆炸+订阅/批量读写粒度失控——区是维度不是类型）？

Q18.8 属性订阅的完整闭环走哪条线？

【深度解答】启动期：ICarImpl.init→VehicleHal.init（VehicleHal.java:218）：fetchAllPropConfigs（:182 定义/:219 调用）拉全表→对每个 HalServiceBase 按 getAllSupportedProperties/isSupportedProperty 匹配（:229 起），命中即 mPropertyHandlers.append(propId, service)（:235/:245）→takeProperties（:256）→service.init()（:257）。运行期闭环：HalService 调 subscribeProperty（:342/:355）——config 为 null 抛 IllegalArgumentException（:369-371）、isPropertySubscribable 校验通过后组装 SubscribeOptions（areaIds+sampleRate :376-378）→mHalClient.subscribe（:379）下到 VHAL。回程：VHAL unsol→onPropertyEvent（:663）按 mPropertyHandlers 路由 →service.onHalEvents(dispatchList)（:686）→CarXxxService 转 CarPropertyManager 回调给 App。〔册16・二・补〕

【追问链】→ set 与 subscribe 的语义差（set 是"请求"不是"状态"——以 unsol 回读为准，写后忙等轮询是反模式）？→ 属性路由表为什么是单射（assertServiceOwnerLocked :329——一属性一个 owner，否则事件分发二义）？→ VHAL 进程死掉闭环怎么自愈（init 重走 configs 分发+各 service 重订阅——CarService 层面的 eventually 一致）？

第十九章 无障碍体系题（10 题）

Q19.1 a11y 服务为什么权限极高？

【深度解答】三层能力：读屏（AccessibilityNodeInfo 全部内容含密码框）、合成操作（performAction 替用户点击/输入）、输入截胡（AccessibilityInputFilter:59 可改写/吞事件）。恶意

a11y=后门。13 起禁 sideload 静默启用。〔册17・一/四〕

【追问链】→ 为什么不能只给"读屏"不给"操作"（开关控制用户需要代理操作）？ → 安全防护三层（安装来源限制+用户弹窗+事件过滤策略）？ → TalkBack 为什么不被滥用（Google 签名+开源审查）？

Q19.2 AccessibilityNodeInfo 是什么协议?

【深度解答】View 树的"语义快照"：a11y 服务不能直接操作 View 对象（跨进程）——通过快照（文本/类名/位置/可点击/子节点）了解界面，再 performAction 回到 View。快照保证原子一致（读到那一刻的完整界面）。〔册17・三〕

【追问链】→ 自定义 View 怎么适配（覆写 onInitializeAccessibilityNodeInfo）？ → 快照的生成时机（performAccessibilityAction 触发）？ → 为什么不用 Binder 直传 View（安全+一致性）？

Q19.3 AccessibilityInputFilter 在输入链的哪个位置?

【深度解答】书5 输入链的**最后一个消费者**：InputDispatcher→ViewRootImpl InputStage→a11y InputFilter（:59 extends InputFilter implements EventStreamTransformation）——服务可改写/吞/合成事件（开关控制的"扫描→按开关→合成点击"走 :364 onMotionEvent）。〔册17・二〕

【追问链】→ 触摸探索为什么接管全部触摸（先触摸播报再双击确认——第一步必须截获否则被普通点击消费）？ → 合成的手势走什么通道（GestureDescription→dispatcher 合成入口）？ → a11y 截输入会误伤普通用户吗（a11y 服务启用才有 filter）？

Q19.4 无障碍服务的绑定链里藏了哪些特权 flags?

【深度解答】用户启用服务即触发绑定：AccessibilityServiceConnection（AccessibilityServiceConnection.java:63）.bindLocked（:105）以一组越权 flags 调 bindServiceAsUser（:117-119）：BIND_AUTO_CREATE（常驻）、BIND_FOREGROUND_SERVICE_WHILE_AWAKE（亮屏时不按后台杀）、BIND_ALLOW_BACKGROUND_ACTIVITY_STARTS（可从后台拉起 Activity——fullscreen 类能力）、BIND_ALLOW_INSTANT（Instant 服务，:114-116 开关）、BIND_INCLUDE_CAPABILITIES（继承调用方能力）。绑定前置 Binder.clearCallingIdentity（:107-108）；先占位 bindingServices（:120）防重入；成功回调 onServiceConnected（:170）接通服务。unbindLocked（:129）对称清理并把 setAllowAppSwitches 置回 -1（:136-138）。〔册17・一・补〕

【追问链】→ 为什么敢给 BIND_FOREGROUND_SERVICE_WHILE_AWAKE（读屏服务被杀=视障用户界面全瞎，可用性优先于省电）？ → 绑定失败/超时怎么收尾（bindingServices 占位超时移除，服务列表回归一致）？ → setAllowAppSwitches 的用途（a11y 服务代用户切应用时的合法身份，unbind 归零防权柄残留）？

Q19.5 一条 a11y 事件从 App 到服务要过几道安全闸?

【深度解答】三道。闸一 AMS 入口 sendAccessibilityEvent（AccessibilityManagerService.java:906）：resolveCallingUserIdEnforcingPermissionsLocked（:928）钉死 userId、resolveValidReportedPackageLocked（:931-933）把包名钉到调用方 uid——伪造包名的事件在此拍平；canDispatchAccessibilityEventLocked（:937）决定放行（input filter 旁路 :1003）。闸二连接级过滤 notifyAccessibilityEvent（AbstractAccessibilityServiceConnection.java:1472）：wantsEventLocked 按服务声明的事件类型筛选（:1476）、checkAccessibilityAccess 查用户级开关（:1483）——不感兴趣直接回收，连 copy 都省。闸三派发前 notifyAccessibilityEventInternal（:1548）：canRetrieveWindowContentLocked 不过则 event.setSource(null)（:1589-1594）剥掉内容引用，随后 event.setSealed(true)（:1594）封板过 binder。〔册17・二・补〕

【追问链】→ 闸二通过后为什么还要 copy 事件（闸三要按权限改写它——可变对象不能直接给原引用）？→ mNotificationTimeout 节流为什么豁免 TYPE_WINDOW_CONTENT_CHANGED（:1490-1499——内容变更要即时供 AccessibilityCache，节流反而饿死缓存）？→ 包名钉死挡的是什么攻击（恶意 App 假冒系统包发事件诱导读屏服务/钓鱼）？

Q19.6 节点快照的 seal 协议怎么保证"快照不可篡改"?

【深度解答】AccessibilityNodeInfo 自带封板位：字段 mSealed（AccessibilityNodeInfo.java:852）、setSealed（:3653）、isSealed（:3664-3665）；几乎全部 mutator（setText/addChild/setParent/setClickable……）先查 sealed，封板后修改抛 IllegalStateException（:3676/:3714 两处守卫）。跨进程一致性靠序列化：sealed 标志随 parcel 写出（:3942 writeInt(isSealed)?1:0）——服务端解包出来的树天然只读。事件侧同样在派发前 setSealed(true)（AbstractAccessibilityServiceConnection:1594）。于是"读=拿那一刻的合同"，"写=performAction 回源进程由 View 重校验执行"——读路径零信任，写路径集中管控。〔册17・三・补〕

【追问链】→ 为什么不能靠服务"自觉只读"（AccessibilityCache 全局共享同一对象图——一处篡改污染所有读者与后续查询）？→ uiautomator（UiAutomation）拿的节点也 sealed 吗（同走 a11y 连接，同样封板——自动化工具无特权旁路）？→ 服务 A 的节点对象传给服务 B 会怎样（connectionId/seed 校验不匹配，B 查不到内容——快照绑定连接）？

第二十章 CTS 认证题（10 题）

Q20.1 CTS 通过意味着什么?

【深度解答】拿到 GMS（Google Play）预装权——设备厂商的"营业执照"。CTS 约 200-300 万断言，跑完 8-72 小时；分片可并行。〔册18・一〕

【追问链】→ 不过 CTS 能卖吗（能——但没 Play Store）？→ CTS 每 Android 版本更新吗（是）？→ 谁跑 CTS（设备厂商，Google 审结果）？

Q20.2 CTS/VTS/GTS 三者覆盖面?

【深度解答】CTS：framework API 行为（App 视角）；VTS：HAL 接口（书14 的 AIDL HAL 逐个验证）；GTS：Google 专有（GMS 验证，闭源发行）。三者都过=完整认证。〔册18・三〕

【追问链】→ VTS 测什么（HAL 方法的返回值/超时是否符合 .aidl 定义）？→ 厂商 HAL 改了忘了过 VTS 的后果（CTS 连锁 fail）？→ Tradefed 是共用的吗（是——同一调度引擎跑三种套件）？

Q20.3 Tradefed 的执行流水线?

【深度解答】四段：TestInvocation 入口→DeviceMonitor（选设备/监控状态）→TestRunner（逐用例跑+截图/日志）→ResultReporter（聚合 XML/HTML）；RetryFilter 对已知 flaky 重跑。〔册18・二〕

【追问链】→ 分片（shard）怎么工作（多台设备各跑一部分模块）？→ fail 结果怎么看（testResult.xml+截图+logcat）？→ waiver 是什么（Google 审批的用例豁免）？

Q20.4 一个 CTS 模块的"三件套"各承载什么?

【深度解答】以 cts/tests/tests/permission2 为例。①Android.bp：android_test（:21）声明 name CtsPermission2TestCases（:22）、test_suites ["cts","general-tests"]（:25-28）——决定编出测试 APK 并打进哪些套件镜像；②AndroidManifest.xml：instrumentation 声明（目标包+runner）；③AndroidTest.xml：tradefed 模块配置——test-suite-tag cts（:18）、not-shardable（:19）、config-descriptor metadata（component :21、parameter 变体位、token SIM_CARD : 26 的调度约束）、SuiteApkInstaller 装 APK（:28-30）。三件套各答一问：能编译吗（bp）、能装吗（manifest）、能被调度执行吗（xml）——三者齐备，构建期 CtsConfigLoadingTest 校验配置才放行。〔册18・二・补〕

【追问链】→ hostsidetests 模块的差异（宿主侧跑 DeviceTestCase，xml 多一段 host runner+推 APK 的 preparer）？→ parameter（instant_app/secondary_user 等）标了会怎样（同一模块按变体各跑一遍，报告分行）？→ 为什么有的模块显式 not-shardable（模块内共享设备状态/安装成本高——切碎反而拖慢总时长）？

Q20.5 CTS 的运行链与 shard 机制?

【深度解答】宿主侧：cts-tradefed 启动（cts/tools/cts-tradefed/etc/cts-tradefed 脚本拉起 tradefed）→加载 res/config/cts.xml 全套件配置→按 plan/参数筛出模块清单→Tradefed 调度器把模块清单按 shard 逻辑均分到 N 台设备（粒度=模块；AndroidTest.xml 标 not-shardable 的整块走；cts-common.xml:22 注释明言单/多设备同计划混跑）；token 约束（SIM_CARD 等）让无相应硬件的 shard 领不到对应模块。每模块执行：SuiteApkInstaller 装 APK→instrumentation 逐用例→reporters 落 test_result.xml+截图+logcat。总时长=最慢 shard——长尾模块决定全程。〔册18・二・补〕

【追问链】→ shard 数超过设备数会怎样（排队串行——shard 是逻辑划分，并行度受设备数上限）？→ 模块内还能再切吗（默认模块粒度；用例级切分靠参数化变体这类显式声明）？→ 为什么 CTS 强制模块间零依赖（可乱序/可分片/可断点续跑/可 waiver 的前提——依赖即调度死锁）？

Q20.6 retry 会话怎么合并结果？NOT_EXECUTED 从哪来?

【深度解答】`run retry --retry <session>` 走专用配置（cts/tools/cts-tradefed/res/config/retry.xml）：previous_loader 挂 PreviousResultLoader（:17，读上次 session 的 test_result.xml 与失败清单）+RetryRescheduler（:18，只重排 fail/未执行用例的模块）。合并规则：旧 PASS 原样保留、新结果覆盖同名用例、上次没跑到（设备断连/超时/被 shard 丢弃）的保持 NOT_EXECUTED——device 侧同名语义见 CtsVerifier TestResult.java:39 的 TEST_RESULT_NOT_EXECUTED。认证报告允许"多轮 retry 累积到全绿"再提交。device 侧 flaky 自愈另有 util-axt 的 RetryRule（cts/common/device-side/util-axt/…/RetryRule.java）注解级重试。〔册18・二・补〕

【追问链】→ 连续两次 retry 怎么链（第二次的 PreviousResultLoader 读第一次合并后的 session——链式收敛）？→ 为什么 retry 不重跑 PASS（时长不可行+可复现性——PASS 被重跑出 FAIL 反而污染认证结果）？→ waiver 与 retry 的区别（waiver=Google 审批的永久豁免条目；retry=用自证 flaky 换一次重跑机会）？

第二十一章 Telephony 题（10 题）

Q21.1 Telephony 源码为什么在 frameworks/opt?

【深度解答】历史（独立项目起源）+ 演进节奏不同（telephony 每版大改而 core 稳定）+ Mainline 化前置（已部分迁独立 APEX）。GsmCdmaPhone(:141)/RIL(:108)/PhoneFactory(:66) 都在 frameworks/opt/telephony/。〔册19・一〕

【追问链】→ 什么时候迁完 Mainline（进行中）？→ TelephonyManager 在哪（frameworks/base 公共 API 门面）？→ phone 进程是什么（com.android.phone，电话的 system_server）？

Q21.2 RIL 为什么走 socket 不走 Binder?

【深度解答】2009 年设计（Binder 尚未稳定于 native 层）+ rild 是 C 守护进程（socket 更自然）。RIL:108 经 RILSender（Handler 线程）写 socket→rild→厂商 AT/OEM 协议→modem；回复走 RILReceiver。〔册19・二〕

【追问链】→ 未来会改 Binder 吗（低优先——稳定性>优雅）？→ rild 是 AOSP 还是厂商的（AOSP 框架+厂商 vendor 库）？→ RIL 的消息什么格式（原始 AT 或 OEM 自定义协议）？

Q21.3 拨打电话的全链（七跳）？

【深度解答】Dialer→TelephonyManager→**TelecomManager**（调度策略：蓝牙/呼叫等待）→ConnectionService→GsmCdmaPhone.dial→RIL.dial→socket→rild→modem→建立通话→RIL 回调→CallTracker→Telecom→InCallUI。七跳含 Binder+socket 两种 IPC。〔册19・三〕

【追问链】→ TelecomManager 为什么要独立于 Phone（管理多种 ConnectionService——telephony 只是供给方之一）？→ VoLTE 走同一条链吗（ImsPhone 替代 GsmCdmaPhone 段）？→ 蓝牙耳机接听在哪层决定（TelecomManager 的 AudioState）？

Q21.4 APN 数据连接怎么汇入网络栈?

【深度解答】TelephonyProvider（telephony.db）预置 APN→ApnSetting 解析→DataNetworkController 匹配→RIL.setupDataCall→modem 建立→NetworkAgent 供给→汇入书 10 ConnectivityService 评分体系——telephony 是 NetworkFactory 的一个供给方。〔册19・四/书 10 §二〕

【追问链】→ APN 类型（default/mms/ims/supl/…）各自什么时候激活？→ 数据断开重连的状态机（DataConnection 十态）？→ 5G NSA 与 SA 的 DataNetworkController 差异?

Q21.5 VoLTE/IMS 的架构要点?

【深度解答】ImsPhone 实现 Phone 接口但走 IMS 栈（SIP 信令+RTP 语音）——语音从 CS（电路交换）迁到 PS（包交换），打电话不再占"电话信道"而走数据网。运营商 VoLTE 开通=IMS 注册成功。〔册19・五〕

【追问链】→ VoLTE 通话中 4G 数据还能用吗（能——PS 域共享）？→ 为什么有些设备显示"VoLTE"有些不显示（IMS 注册态 UI）？→ ViLTE（视频）与 VoLTE 关系（同一 IMS 域不同媒体类型）?

Q21.6 RIL 的 serial 配对机制怎么保证响应不串线?

【深度解答】请求侧：RILRequest.sNextSerial AtomicInteger（RILRequest.java:41）单调自增、到 Integer.MAX_VALUE 回绕（:99-100）；obtainRequest 领号（RIL.java:1171）后以 mSerial 为 key 挂进 mRequestList（SparseArray :182，:1165-1167），RILSender 写 socket 下发。响应侧：RILReceiver 收 sol 消息→processResponse 三代重载（:5127/:5140/:5155）→processResponseInternal（:5161）按 serial findAndRemoveRequestFromList（:5183）精确取回请求对象分发；对不上号直接 "Unexpected response! serial:" 丢弃（:5185）。serial 是 sol 的"回单号"——响应乱序/交错天然可配对；unsol 无单号，走 registration 表路由。超时兜底 writeOnRilTimeoutResponse（:344）。〔册19・二・补〕

【追问链】→ modem 重启时挂在 mRequestList 的请求怎么收尾（重启路径遍历 mRequestList 逐笔处理 :310-316，上层按 RADIO_NOT_AVAILABLE 兜底重试）？→ 多卡时 serial 会串吗（每 Phone 一个 RIL 实例、独立 serial 空间与 socket）？→ 为什么不用 request id 配对（同 request 并发多笔——如两个 getDataCallList——只有 serial 全局唯一）？

Q21.7 ServiceStateTracker 的双引擎：unsol 闹钟+poll 对账?

【深度解答】引擎一（事件驱动）：modem 主动推——registerForRadioStateChanged（ServiceStateTracker.java:679）、EVENT_NETWORK_STATE_CHANGED（:239，注册态变化 unsol）、setOnNITZTime（:680/:245）。引擎二（轮询对账）：EVENT_POLL_STATE_* 家族（CS/PS 注册 :240-242、operator :243、网络选择模式 :245、入口 :284），pollState（:3346）→pollStateInternal(modemTriggered)（:3350）发起一组查询，mPollingContext（:194）作计数器聚合多路响应，handlePollStateResultMessage（:2214）逐路落账（CS :2284/PS :2294）。设计逻辑：unsol 快但可能丢/迟到，poll 慢但保证收敛——radio on（EVENT_RADIO_STATE_CHANGED :

1332）等分支都触发 poll 兜底，两路汇入同一状态推导。〔册19・三・补〕

【追问链】→ poll 期间 unsol 同时到达会怎样（幂等比较更新——状态没变不刷 UI 不广播）？→ 为什么一次 poll 要发多路查询（CS/PS/operator/网络模式各一个 EVENT_POLL_STATE_*，mPollingContext 计数齐了才继续）？→ NITZ 走哪条引擎（独立 unsol 通道 :680——时间不进注册状态机）？

Q21.8 多卡的 slotIndex/phoneId/subscriptionId 三套索引怎么分工?

【深度解答】三层生命周期不同：slotIndex=卡槽（物理或 eSIM 逻辑槽，UiccController.mUiccSlots :147——插拔维度）；phoneId=逻辑 Phone 实例下标（PhoneFactory 建的 Phone[]——进程内对象维度，DSDS 下与 slot 对齐）；subscriptionId=订阅记录（SubscriptionController 数据库主键——账户维度，换卡生成新 id、旧 id 失效）。映射 API 三件套：getSubId(slotIndex)（SubscriptionController.java:2626）、getSlotIndex(subId)（:2587）、getPhoneId(subId)（:2674）；UiccController.getUiccSlotForPhone(phoneId)（:373）反向找槽。经典坑：getSubId 传非法 slot 会回落默认订阅（:2635 slotIndex=getSlotIndex(getDefaultSubId())）——双卡 App 在此翻车最多。〔册19・二・补〕

【追问链】→ 双卡切默认数据时三套索引谁动谁不动（subscriptionId 换默认，slot/phone 纹丝不动——sim 卡还在原位）？→ eSIM 的 slotIndex 怎么编（eUICC 逻辑槽占高位——同一物理芯片多 profile 轮换）？→ 为什么不统一一套索引（三层生命周期不同：槽随卡、Phone 随重启、订阅随策略——强行合一任何一层变化都污染其余）？

附录 面试策略五条

- **追问链是主考法**：每个答案预留两个"下钻点"，主动说出"这背后还有一个细节是……"引导到你熟的领域。
- **锚点背 20 个**：十跳、九阶段、三批服务、15+1/31、1MB-2页、5s/10s/20s、:824/:763/:3546……锚点脱口而出即"真读过"。
- **画图优先**：启动时序、Binder 十跳、窗口树、输入链路——白板画图比叙述快且可信。
- **知边界**：答"驱动源码不在本树、由 UAPI 反推"这种诚实声明，比编造内核细节加分。
- **把六册当弹药库**：本册每题的〔册・章〕即回查坐标——面试前夜重读各册设计决策章即可焕新。